

Técnicas de búsqueda y heurísticas

Diplomado en Inteligencia Artificial

junio 2025

Profesor: Ricardo Contreras A.

En esta clase

- **Búsqueda no parcial**
 - Método de Montecarlo
 - Aprendizaje con refuerzo
 - Algoritmos basados en poblaciones
 - Algoritmos adaptativos y aprendizaje
- Algoritmos genéticos
- Colonia de abejas
- Colonia de hormigas

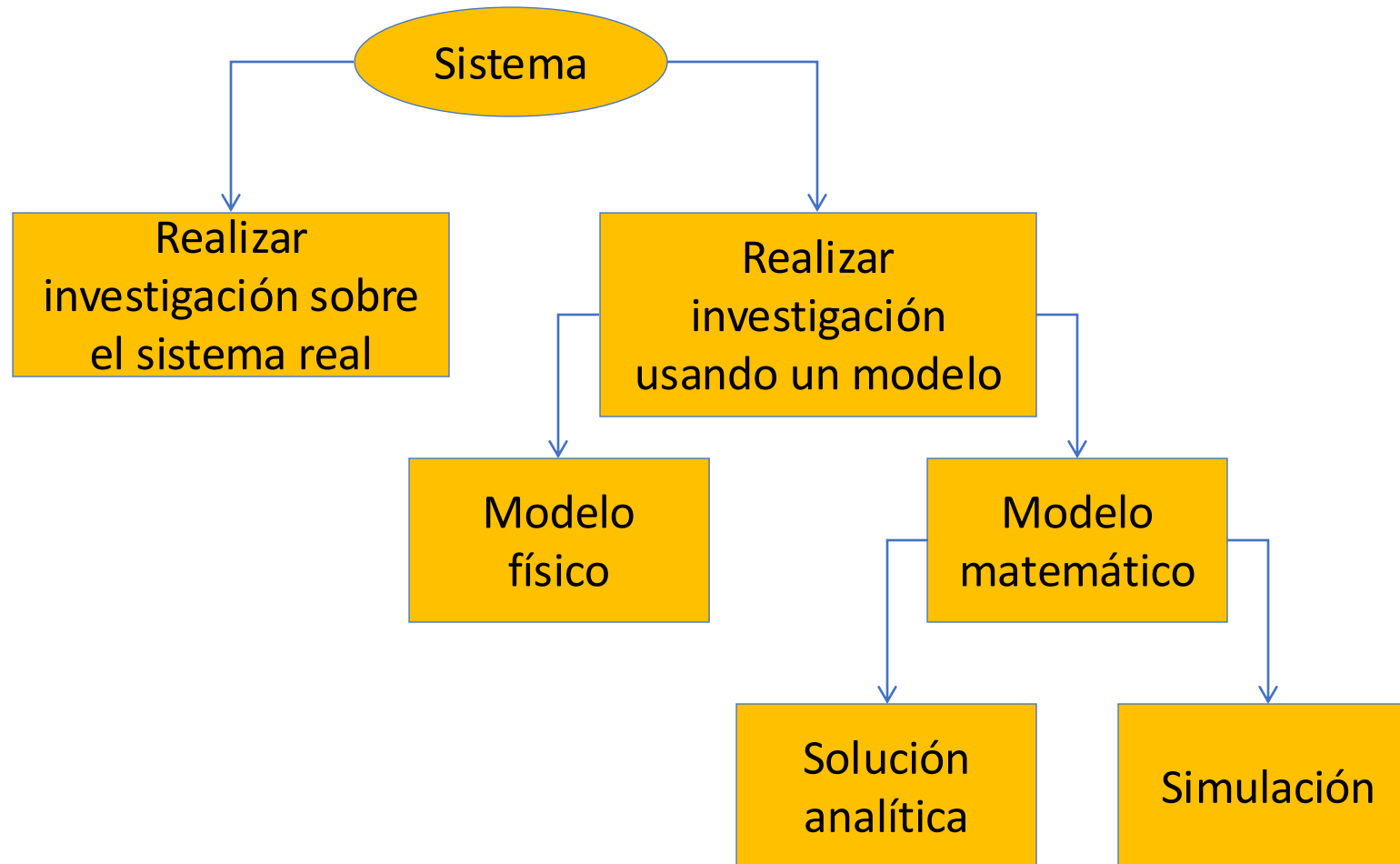
Método de Monte Carlo

- El método de Monte Carlo es una simulación que se utiliza para modelar la probabilidad de diferentes resultados en un proceso que no se puede predecir fácilmente debido a la intervención de variables aleatorias.
- Es una técnica utilizada para comprender el impacto del riesgo y la incertidumbre.

Simulación

- Gracias al desarrollo de las capacidades computacionales, este método es usado para tratar desde la simulación de fenómenos físicos hasta la obtención anticipada de los resultados de la lotería. ¡En teoría!

Simulación



Simulación

- A diferencia de la programación lineal, que es una técnica de optimización, la simulación no determina (necesariamente) la solución óptima.
- Posibilita, a partir del análisis de los experimentos, el análisis del comportamiento del sistema.

Generación de eventos aleatorios

- La generación de eventos aleatorios es una técnica bastante usada en simulación.
- Proceso aleatorio: tablas, ruletas, datos, computadores, etc.
- Inicialmente es necesario identificar las frecuencias del evento a través del examen de datos históricos.

- Cuando existe una incertidumbre importante al hacer una estimación, algunos métodos reemplazan la variable incierta por un solo número promedio.
- En cambio, la simulación de Monte Carlo usa valores múltiples y luego promedia los resultados

Método de Monte Carlo

Combina conceptos estadísticos (muestreo aleatorio) con la capacidad de generar números pseudo-aleatorios y automatización del cálculo.



La invención del método de Monte Carlo se asigna a Stan Ulam y a John von Neuman. Ulam ha explicado cómo se le ocurrió la idea mientras jugaba un solitario durante una enfermedad.

Advirtió que resulta mucho más simple tener una idea del resultado general del solitario haciendo pruebas múltiples con las cartas y contando las proporciones de los resultados que computar todas las posibilidades de combinación formalmente.

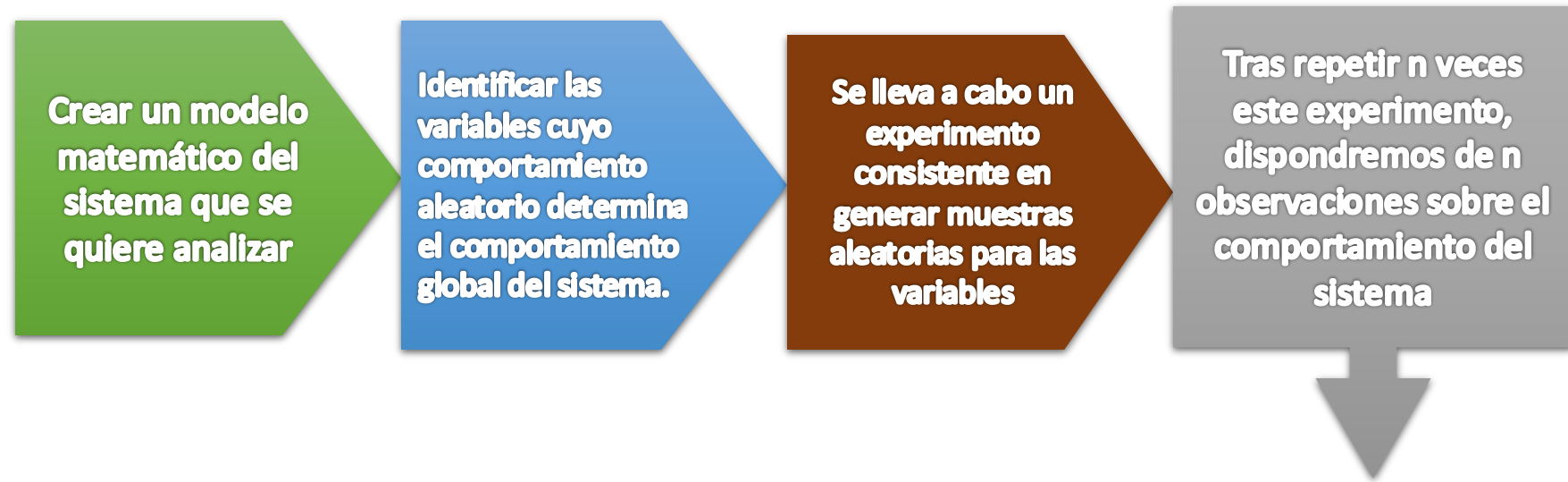
Definición

- La simulación de Monte Carlo es una técnica cuantitativa que hace uso de la estadística en combinación con el poder de los computadores para imitar, mediante el uso de modelos matemáticos, el comportamiento aleatorio de sistemas reales no dinámicos.
- Cuando el estado de los sistemas cambia en el tiempo, se puede recurrir a la simulación de eventos discretos o a la simulación de sistemas continuos.

El método de Monte Carlo es un método no determinístico o estadístico numérico usado para aproximar expresiones matemáticas complejas y costosas de evaluar con exactitud.

El método de Monte Carlo proporciona soluciones aproximadas a una gran variedad de problemas matemáticos posibilitando la realización de experimentos con muestreos de números pseudoaleatorios en una computadora.

“Itinerario” del método



El análisis es más preciso mientras mayor sea n , el número de elementos considerados

Aplicaciones

Se asocian intervalos de números aleatorios según las probabilidades de ocurrencia de los eventos a simular

- Criptografía
- Densidad y flujo de tráfico
- Diseño de reactores nucleares
- Ecología
- Econometría
- Evolución estelar
- Física de materiales
- Pronóstico de valores bursátiles
- Prospecciones petrolíferas
- Radioterapia contra el cancer
- Teoría de filas
- Valoración cartera de valores
- Etc...

La simulación de Monte Carlo se aplica a una infinidad de ámbitos como alternativa a los modelos matemáticos exactos o incluso como único medio de estimar soluciones para problemas complejos.

Así, en la actualidad es posible encontrar modelos que hacen uso de simulación MC en las áreas informática, empresarial, económica, industrial e incluso social.

En otras palabras, la simulación MC está presente en aquellas áreas en la que los sistemas presentan un comportamiento aleatorio o probabilístico.

Jugando con dados

- Para calcular la probabilidad de obtener una suma específica con dos dados, se pueden simular varias tiradas mediante la generación de números aleatorios.
- Por cada tirada simulada, se generan dos números aleatorios entre el 1 y el 6 (que representan las caras del dado).
- Al repetir este proceso varias veces, se pueden contar las veces que se obtiene la suma deseada y estimar su probabilidad.

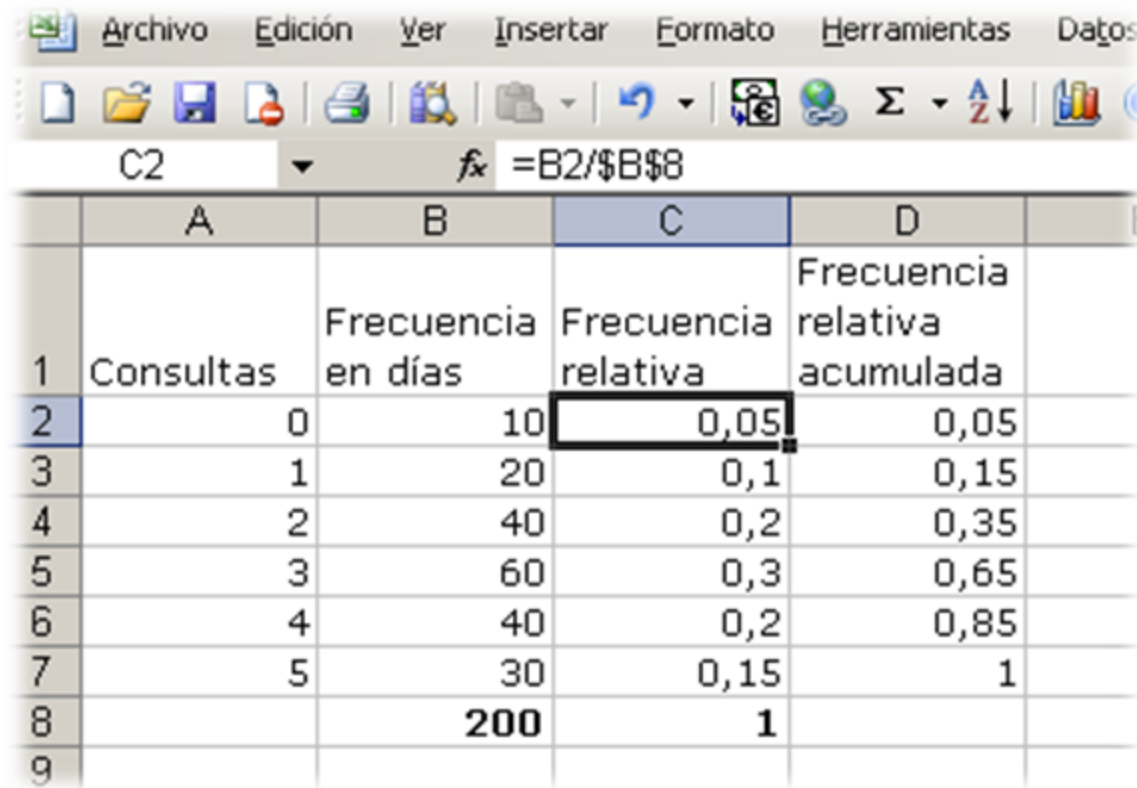


Ejemplo

Una forma de hacer pruebas de Monte Carlo es con una hoja de cálculo como Microsoft Excel.

Supongamos que se tiene un análisis histórico de 200 días sobre consultas realizadas en un sistema de información.

La tabla muestra el número de consultas diarias (de 0 a 5) junto con las frecuencias absolutas (# de días por cada frecuencia), las frecuencias relativas y las frecuencias relativas acumuladas.



The screenshot shows the Microsoft Excel interface with the menu bar (Archivo, Edición, Ver, Insertar, Formato, Herramientas, Datos) and the toolbar. The active cell is C2, and the formula bar shows the formula $=B2/\$B\8 . The table below represents the data being analyzed:

	A	B	C	D
	Consultas	Frecuencia en días	Frecuencia relativa	Frecuencia relativa acumulada
1				
2	0	10	0,05	0,05
3	1	20	0,1	0,15
4	2	40	0,2	0,35
5	3	60	0,3	0,65
6	4	40	0,2	0,85
7	5	30	0,15	1
8		200	1	
9				

Podemos interpretar la frecuencia relativa como la probabilidad de que ocurra el suceso asociado, en este caso, la probabilidad de un determinado número de consultas (así, por ejemplo, la probabilidad de que se den 3 consultas en un día sería de 0,30), por lo que la tabla anterior nos proporciona la distribución de probabilidad asociada a una variable aleatoria discreta (la variable aleatoria es el número de consultas, que sólo puede tomar valores enteros entre 0 y 5).

Supongamos que queremos conocer el número esperado (o medio) de consultas por día.

Una forma directa es haciendo la operación

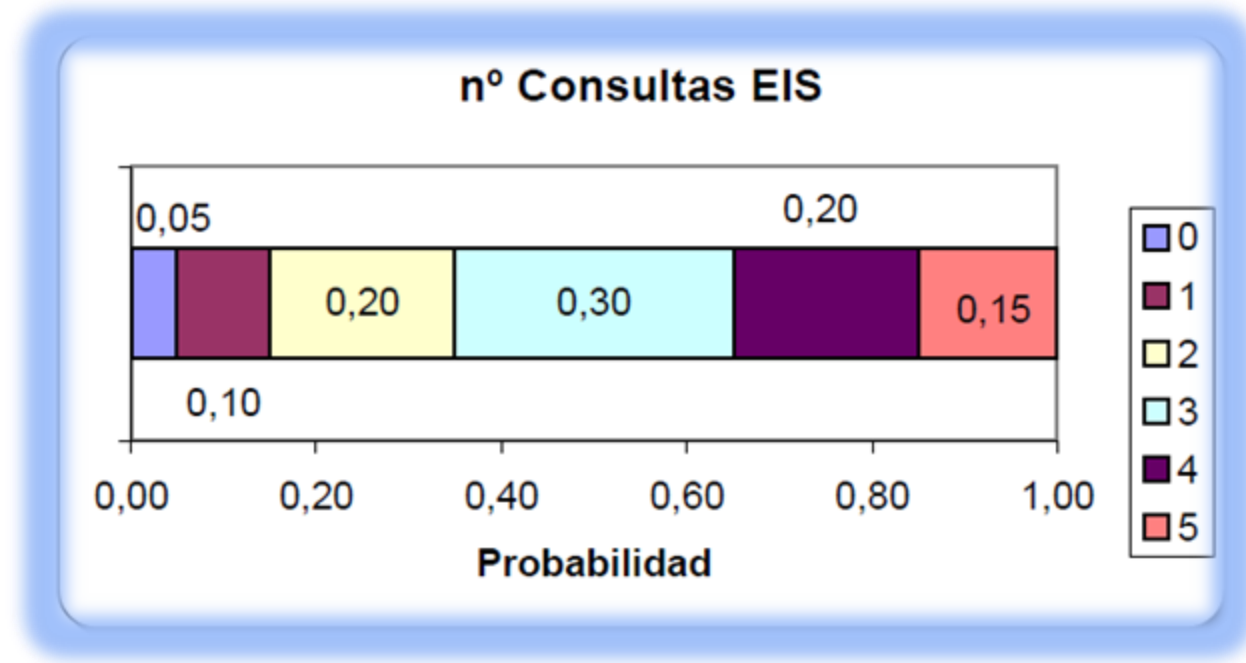
$$\text{Valor Medio} = \text{sumatoria} (\# \text{de visitas} * \text{Probabilidad de que ocurran}) = 0*0,05 + 1*0,1 + 2*0,2 + 3*0,3 + 4*0,2 + 5*0,15 = 2,95$$

Por otro lado se puede usar una simulación Monte Carlo para deducirla. Para ello se tiene en cuenta las frecuencias relativas acumuladas de esta manera:

[0,00 a 0,05) para el suceso 0
[0,05 a 0,15) para el suceso 1
[0,15 a 0,35) para el suceso 2
[0,35 a 0,65) para el suceso 3
[0,65 a 0,85) para el suceso 4
[0,85 a 1,00) para el suceso 5

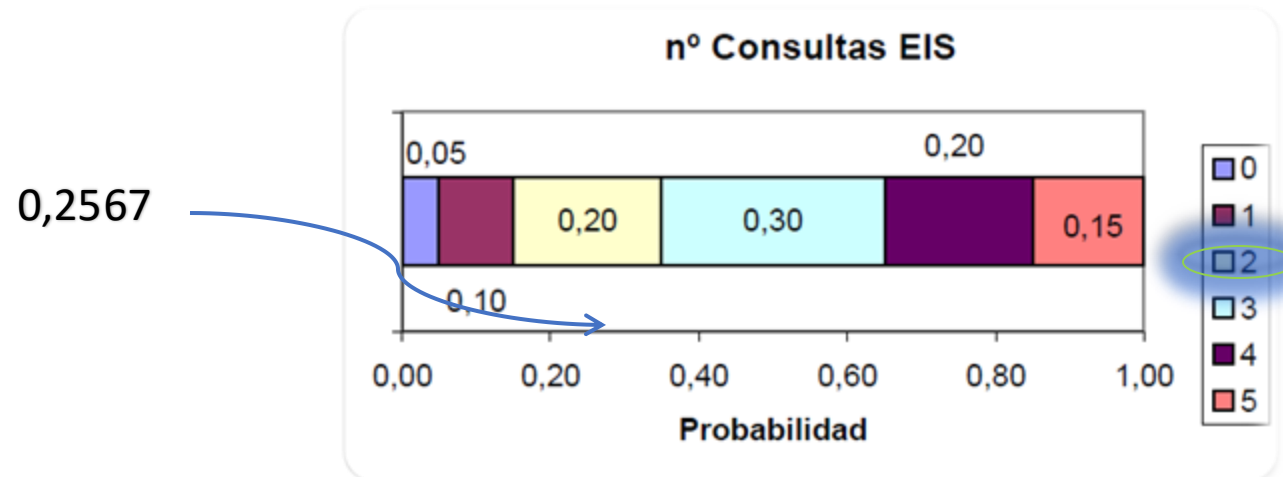
El gráfico muestra cada una de las probabilidades sobre el número de consultas.

Se muestra la relación entre probabilidad de cada suceso y el área que éste ocupa.



Esto significa que, al generar un número pseudo-aleatorio (proveniente de una distribución uniforme entre 0 y 1), estaremos llevando a cabo un experimento cuyo resultado, obtenido de forma aleatoria y según la distribución de probabilidad anterior, estará asociado a un suceso.

Así por ejemplo, si el computador nos entrega el número pseudo-aleatorio 0,2567, podremos suponer que ese día se han producido 2 consultas



A veces la aplicación del método Monte Carlo se usa para analizar problemas que no tienen un componente aleatorio explícito.

En estos casos, un parámetro determinista del problema se expresa como una distribución aleatoria y se simula dicha distribución.

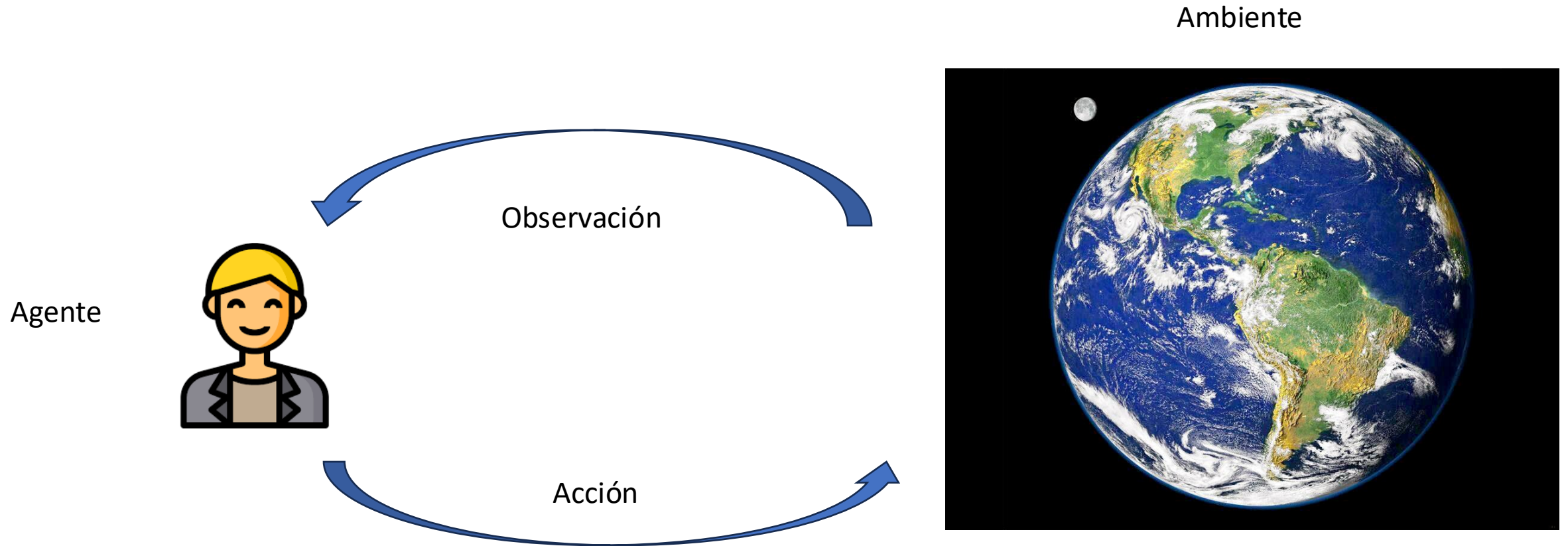
Reinforcement Learning



Aprendizaje con refuerzo (Reinforcement learning)

- ¿Qué es?
- Personas y animales aprenden por **interacción con el ambiente**
 - Esto difiere de otros tipos de aprendizaje
 - Es **activo** en vez de pasivo
 - Las interacciones son a menudo **secuenciales** — interacciones futuras pueden depender de interacciones previas
- Estamos dirigidos por **objetivos**
- Podemos aprender **sin ejemplos** de comportamiento optimal
- Optimizamos algún tipo de **recompensa**

El ciclo interactivo



Meta: optimizar la suma de recompensas a través de interacciones repetidas

La hipótesis de la recompensa

- El aprendizaje con refuerzo se basa en esta hipótesis:
 - *Cualquier meta puede ser formalizada como el resultado de maximizar las recompensas acumuladas (el “retorno”)*

Ejemplos

- Acción

- Volar un aeroplano
- Manejo de inversiones
- Caminata de un robot
- Jugar juego de video

- Recompensa

- Tiempo de vuelo, distancia recorrida
- Ganancias, disminución del riesgo
- Distancia, velocidad
- Ganar, maximizar puntaje

Si la meta es aprender a través de la interacción, todos estos son problemas de aprendizaje con refuerzo (con independencia de la solución)

¿Por qué aprendizaje con reforzamiento?

- Hay distintas razones para aprender

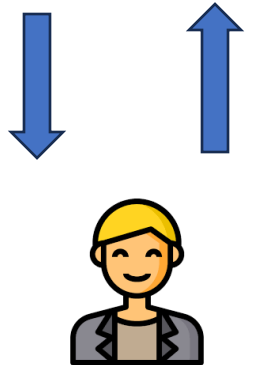
- Encontrar soluciones
 - Un programa que juegue muy bien ajedrez
 - Un robot que trabaje sobre un producto de manufactura específico
- Adaptación a los cambios (“online”). La reacción ante cambios imprevistos.
 - Un programa que juega ajedrez que se adapta a un usuario
 - Un robot capaz de “navegar” en terrenos desconocidos
- Para ambos casos se pueden diseñar algoritmos
- En el caso de adaptación a los cambios, no se trata (solamente) de generalización. Se trata de seguir aprendiendo (eficientemente) **durante** la operación

¿De qué se trata?

- Conjugar aspectos de ciencia y un marco para aprender a tomar decisiones a partir de la interacción
- Esto requiere reflexionar sobre
 - ... tiempo
 - ... consecuencias (a largo plazo) de las acciones
 - ... recopilación de experiencias
 - ... proyecciones (de futuro)
 - ... manejo de incertidumbre

Pensemos en un juego (¿de Atari?)

- En cada paso t , el agente
 - Recibe la observación O_t (y la recompensa R_t)
 - Ejecuta la acción A_t
- Y en cada paso t , el ambiente
 - Recibe la acción A_t
 - Emite la observación O_{t+1} (y la recompensa R_{t+1})



Recompensas

- Una **recompensa** R_t es una señal de realimentación escalar
- Indica qué tan bien lo está haciendo el agente en el paso t (mirando al objetivo)
- El trabajo del agente consiste en maximizar la ganancia acumulada
 - $G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$
- Esto es lo que se llama **retorno**
- **Recordemos**
El aprendizaje con reforzamiento se basa en la **hipótesis de la recompensa**

El valor

- Se le llama **valor** a la recompensa acumulada esperada, para un estado **s**

$$v(s) = E[G_t | S_t = s]$$

$$= E[R_{t+1} + R_{t+2} + R_{t+3} + \dots | S_t = s]$$

- El valor depende de las acciones que tomen los agentes
- La meta es **maximizar el valor**, tomando las acciones adecuadas
- Recompensas y valores definen la **utilidad** de los estados y acciones (no hay realimentación supervisada)

Maximizar valores de acuerdo a las acciones

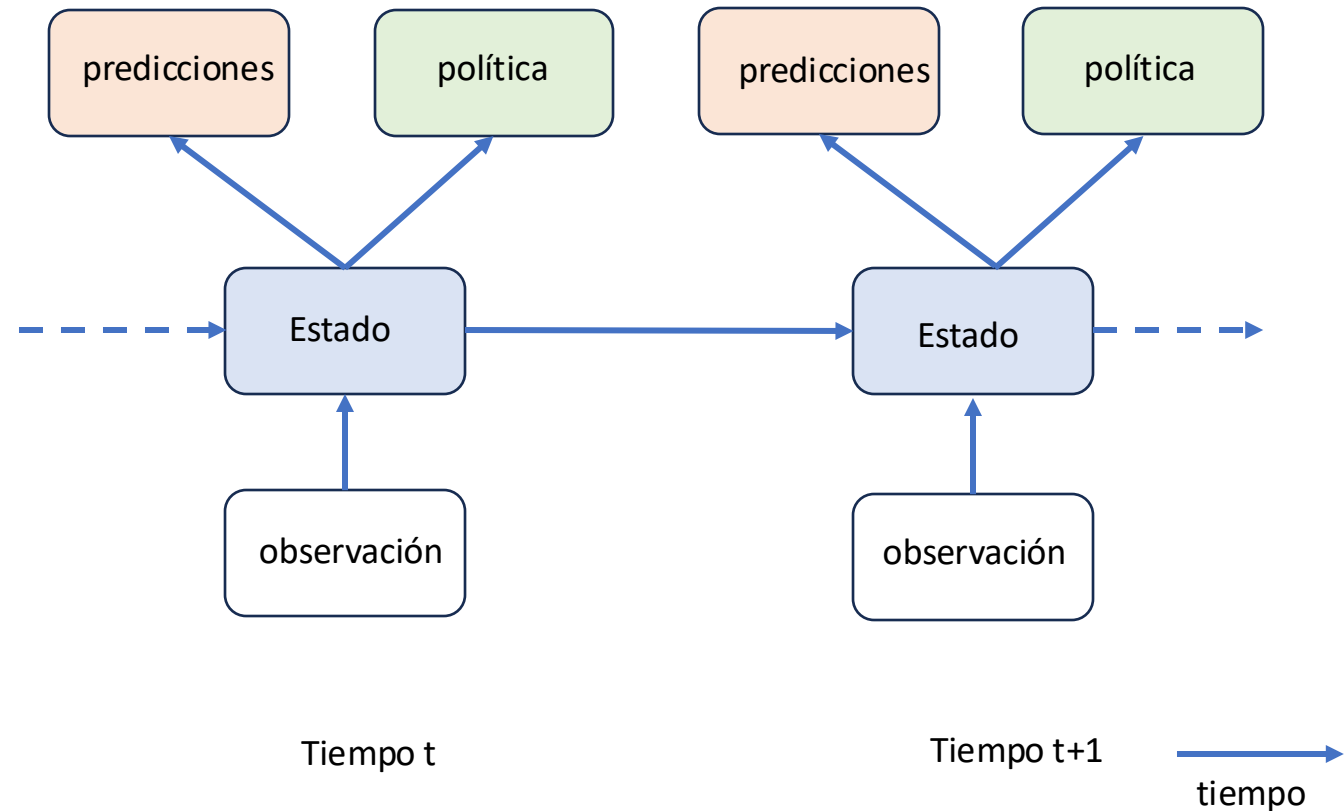
- Objetivo: **seleccionar acciones para maximizar el valor**
- Acciones pueden tener consecuencias en el largo plazo
- Recompensas pueden ser postergadas
- Puede ser mejor sacrificar una recompensa inmediata para obtener una mejor recompensa en el mediano plazo
- Ejemplos:
 - Reabastecer un helicóptero (puede prevenir una caída en varias horas)
 - Jugadas defensivas en un juego (puede mejorar las posibilidades de ganar más tarde)
 - Aprender una nueva habilidad (puede ser de alto costo y tiempo al inicio)
- Un mapeamiento desde estados a acciones de lo que se llama una **política**

El formalismo para el aprendizaje con refuerzo incluye

- **Ambiente** (dinámica del problema)
- **Recompensa** (especifica la meta)
- **Agente**, que considera:
 - Estado del agente
 - Política
 - ¿Estimación del valor de la función?
 - ¿Modelo?

Ahora, vamos con el agente. Componentes

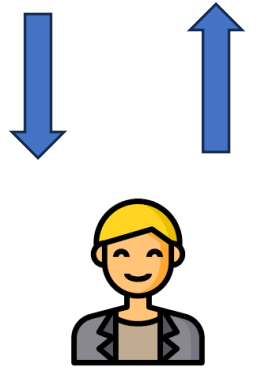
- Estado del agente
- Política
- Valores de las funciones
- Modelo



Estado del ambiente



- El **estado del ambiente** es el estado interno del ambiente
- Normalmente invisible para el agente
- Aún si fuera visible, puede contener mucha información irrelevante



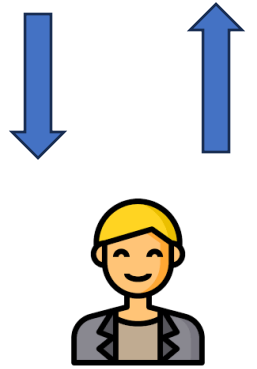
Estado del agente

- La **historia** es la secuencia completa de observaciones, acciones, recompensas
 - $\mathcal{H}_t = O_0, A_0, R_1, O_1, \dots, O_{t-1}, A_{t-1}, R_t, O_t$
- Por ejemplo, el flujo entre los sensores de un robot
- Esta historia se usa para construir el **estado del agente** S_t

Ambientes completamente observables



- Supone que el agente **ve** el estado completo del ambiente
- Observación = estado del ambiente
- El estado del agente podría ser exactamente esta observación:
 - $S_t = O_t = \text{estado del ambiente}$



Ambientes parcialmente observables

- **Observabilidad parcial:** Las observaciones no son Markovianas
 - La cámara de un robot no conoce su ubicación absoluta
 - Un agente que juega póker solo puede ver las cartas que son “públicas”

Política de un agente

- Una **política** define el comportamiento del agente
- Es un mapeamiento desde el estado del agente a una acción
- La política puede ser determinística: $A = (S)$ *(una acción lleva a un estado)*
- La política puede ser estocástica: $(A|S) = p(A|S)$ *(existe una probabilidad)*

Modelo de un agente

- Un **modelo** predice lo que sucederá con el ambiente (a continuación)
 - Por ejemplo, predice el próximo estado
 - Predice la próxima recompensa
- Un modelo no garantiza una buena política. Se necesita una planificación

Subproblemas del aprendizaje con reforzamiento

- Predicción
 - Evaluar el futuro para una política dada
- Control
 - Optimizar el futuro (encontrar la mejor política)
- *Ambos están fuertemente relacionados*

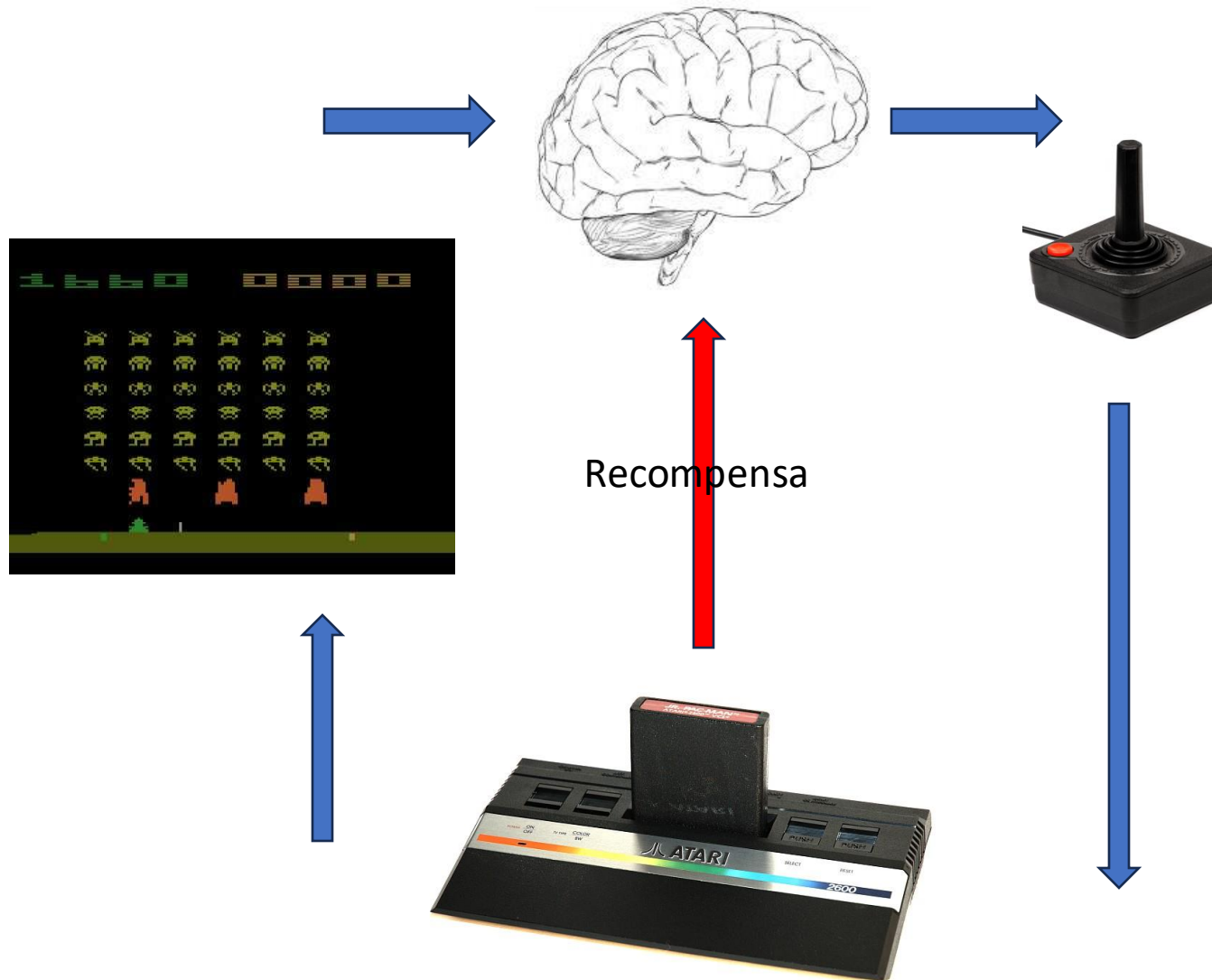
Subproblemas del aprendizaje con reforzamiento (Cont.)

- Aprendizaje
 - El ambiente es desconocido inicialmente
 - El agente interactúa con el ambiente
- Planificación
 - Es necesario tener un modelo del ambiente (o aprenderlo)
 - El agente planifica en este modelo (sin interacción externa)

Componentes del aprendizaje del agente

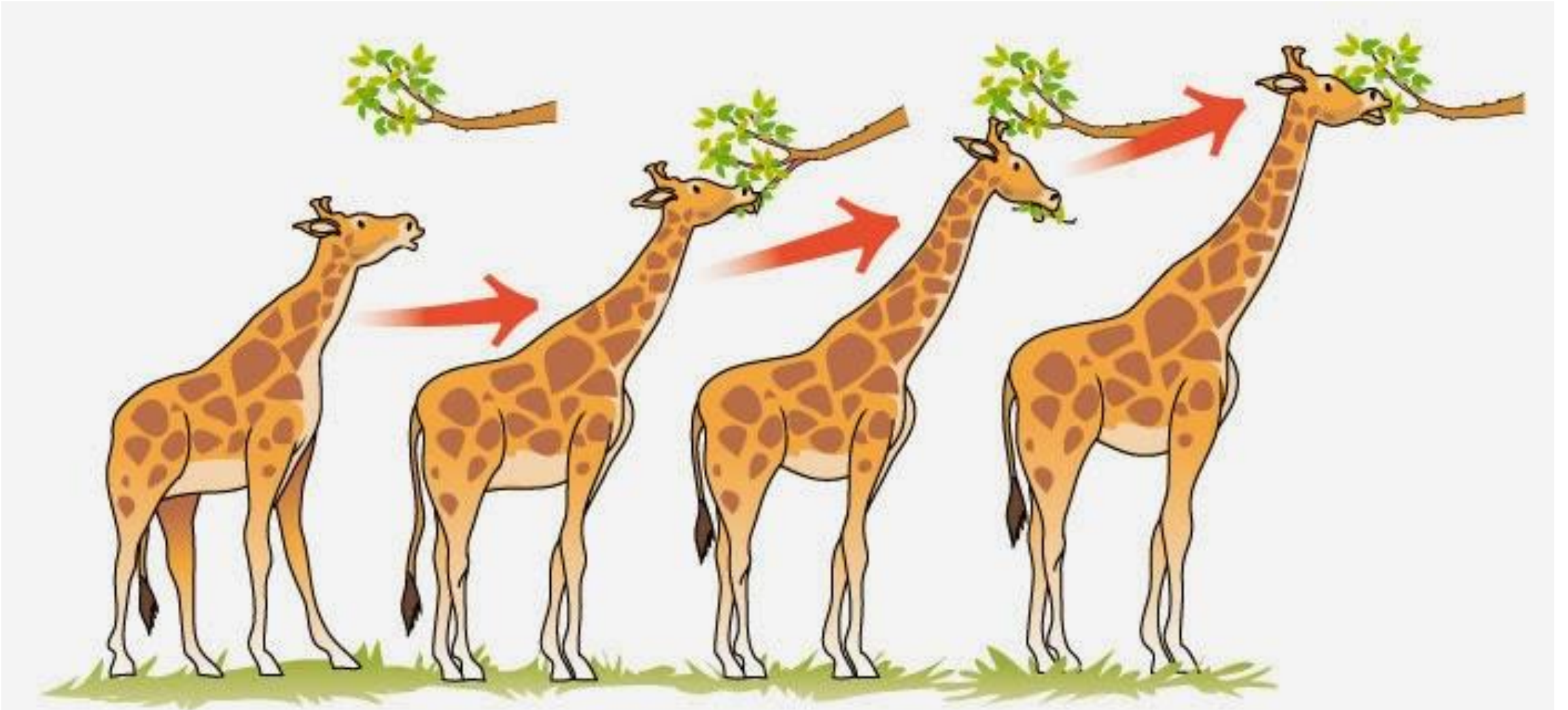
- Todos los componentes son funciones
 - Políticas: $\pi : S \rightarrow A$ (o a una probabilidad de A)
 - Valores de funciones: $v : S \rightarrow R$
 - Modelos: $m: S \rightarrow S$ y/o $r: S \rightarrow R$
 - Actualiza estado: $a: S \times O \rightarrow S$
- En general, se puede usar redes neuronales y usar técnicas de **deep learning** para aprender

Ejemplo con Atari



1. Las reglas del juego son desconocidas
2. Se aprende directamente de la interacción
3. Las acciones sobre el joystick dependen de los pixeles y los puntajes

Algoritmos Adaptativos



¿Qué es un algoritmo adaptativo?

- Un **algoritmo adaptativo** es un procedimiento computacional, o un conjunto de reglas que ajustan de manera dinámica su comportamiento o su estructura, como respuesta a variaciones en los datos de entrada.
- Esto permite que un sistema optimice su operación a través del tiempo.
- A diferencia de los algoritmos convencionales, con parámetros fijos, los algoritmos adaptativos poseen la capacidad única de modificar sus configuraciones de manera autónoma.
- Esta característica es esencial para enfrentar con éxito escenarios impredecibles y que evolucionan en aplicaciones de IA.

Uso

- Son una clase especial de algoritmos diseñados para analizar e interpretar conjuntos de datos complejos.
- Adaptabilidad se refiere a la característica de poder responder a datos o patrones no conocidos alterando su configuración interna.
- Son capaces de su precisión y poder predictivo primero modelando y luego automatizando el proceso de toma de decisiones.
- Su naturaleza adaptativa es lo que les permite (como sistemas AI) mantener su relevancia y efectividad en ambientes dinámicos.

Principios

- Los principios operacionales de los algoritmos adaptativos están asociados a su objetivo central: ajustarse dinámicamente en función de los datos de entrada, refinando su rendimiento y por lo tanto sus capacidades predictivas.

Sus aspectos clave

- **Ajuste dinámico de parámetros:** Los algoritmos adaptativos tienen la capacidad de ajustar sus parámetros internos y configuraciones como respuesta a patrones de datos que evolucionan, lo que permite un desempeño óptimo ante diversos escenarios.

Sus aspectos clave (Cont.)

- **Mecanismos de aprendizaje:** Estos algoritmos están equipados con mecanismos de aprendizaje que les permiten reconocer escenarios y adaptarse a patrones, tendencias, y anomalías en los datos, lo que les permite mejorar su precisión a través del tiempo.

Sus aspectos clave (Cont.)

- **Ciclos de realimentación:** A través de estas realimentaciones, estos algoritmos pueden refinar sus predicciones y decisiones basados en los resultados de sus acciones previas, fomentando un mejoramiento y adaptabilidad de forma iterativa.

Ejemplo 1. Algoritmo adaptativo en predicción financiera

- El rol clave está en la capacidad de analizar la dinámica del mercado, identificar tendencias y adaptar los modelos predictivos a las condiciones económicas rápidamente cambiantes
- Al ajustarse de manera autónoma ante nuevos datos y fluctuaciones del mercado, estos algoritmos pueden realizar una predicción más precisa, por ejemplo informando respecto de decisiones de inversión estratégicas y evaluación de riesgos.

Ejemplo 2. Un algoritmo adaptativo en robótica

- Apuntan a la adecuación de una máquina (el robot) para responder dinámicamente a estímulos ambientales y a obstáculos emergentes.
- En vehículos autónomos, automatización industrial, o manipuladores robóticos complejos, la naturaleza adaptativa de estos algoritmos permite incorporar en las máquinas la capacidad de hacer una sintonía fina de sus acciones en base a entradas en tiempo real. El objetivo es mejorar la precisión y la agilidad.

Rjemplo 3. Marketing personalizado

- En el reino del marketing, y compromiso con los clientes, los algoritmos adaptativos facilitan recomendaciones personalizadas, publicidad dirigida, y modelamiento dinámico de precios.
- Al adaptarse continuamente al comportamiento y preferencias de los usuarios/clientes, estos algoritmos pueden manejar diferentes estrategias de marketing, mejorando la satisfacción y el compromiso de los clientes.

Ventajas y desventajas

- La aplicación de este tipo de algoritmos en ambientes de IA, presenta ventajas y desafíos, al enfrentar diferentes escenarios

Ventajas

- **Resiliencia:** En ambientes dinámicos, se pueden adaptar a condiciones cambiantes y (posiblemente) a eventos inesperados sin intervención humana.
- **Mejoramiento de la potencia predictiva:** Su capacidad para refinar de manera autónoma sus modelos predictivos, gatillado este refinamiento por la aparición de nuevos datos.
- **Evolución:** Estos algoritmos evolucionan continuamente para capturar patrones emergentes y tendencias.

Desventajas

- **Complejidad:** Administrar e interpretar el comportamiento adaptativo puede ser muy complejo. Esto requiere experiencia y gran capacidad de análisis.
- **Sesgo algorítmico:** Se pueden incorporar modelos sesgados a los algoritmos clásicos, lo que requiere estrategias robustas de detección de comportamientos algorítmicos clásicos.
- **Recursos computacionales:** La naturaleza adaptativa demanda recursos computacionales importantes, lo que es un gran desafío en ambientes con recursos limitados.

Tópicos relacionados

- Hay muchos términos y conceptos relacionados con los *algoritmos adaptativos*:
- **Reinforcement Learning:** Un área que se preocupa con el cómo los agentes software deben tomar ciertas acciones en un ambiente para maximizar la noción de recompensa acumulativa.
- **Neural Networks:** Estos son algoritmos que reconocen relaciones subyacentes en un conjunto de datos a través de procesos que están basados en la forma en que opera el cerebro.
- **Algoritmos genéticos:** Una técnica de búsqueda heurística y optimización inspirada en el proceso de la evolución natural.

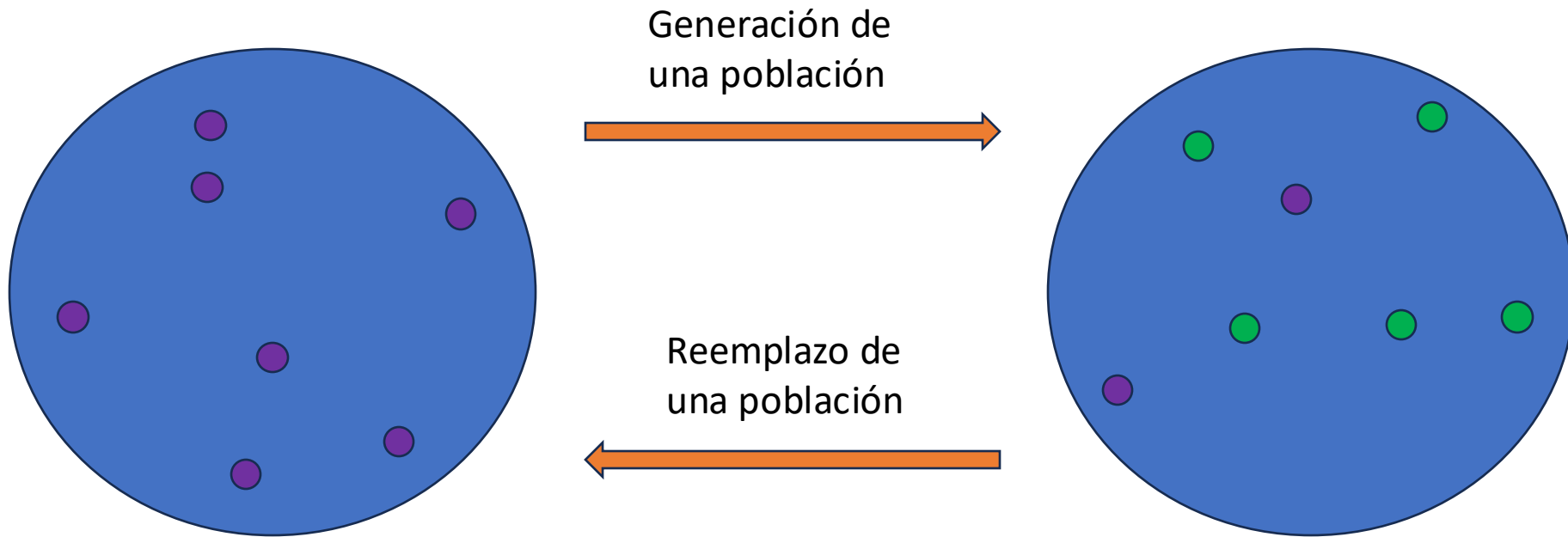
Conclusiones

- El reino de los *algoritmos adaptativos* representa un aspecto central de los sistemas de IA modernos.
- Incorporan la capacidad de aprendizaje autónomo, adaptación y resiliencia frente a ambientes cambiantes.
- Se abrirán, con alta probabilidad, caminos para nuevas miradas en análisis predictivo, toma de decisiones y sistemas autónomos.

Algoritmos basados en poblaciones

- Los algoritmos basados en poblaciones son metaheurísticas que comparten muchos elementos.
- En general se pueden entender como mejoramientos iterativos en una población de soluciones.
 - Primero, se inicializa una población
 - Luego, se genera una nueva población de soluciones
 - La nueva población es integrada en la actual, usando algún mecanismo de selección
 - El proceso de búsqueda se detiene cuando se satisface una condición dada (el criterio de detención)

Principios de una Metaheurística-P



Conceptos básicos

- La **generación y reemplazo** de poblaciones pueden ser de dos tipos:
 - *Sin memoria*; en este caso, los dos procedimientos se basan solamente en la población actual.
 - *Con memoria*; se puede usar la historia almacenada en la memoria para la generación y el reemplazo de una población.
- La mayoría de las metaheurísticas-P usan algoritmos inspirados en la naturaleza. Los ejemplos más populares son optimización por colonias de hormigas, por colonias de abejas, sistemas inmunes artificiales, entre otros.

Ejemplo de template para una metaheurística-P

Estructura de alto nivel para una metaheurística-P

$P = P_0$; /* Generación de la población inicial */

$t = 0$

Repetir

 Generar $P'(t)$ /* Generación de una nueva población */

$P_{t+1} = \text{Seleccionar-Población}(P_t \cup P'(t))$ /* Selección de una nueva población */

$t = t + 1$

Hasta que se satisfaga criterio de detención

Salida: Mejor(es) solución(es) encontrada(s)

Diferencias entre metaheurísticas

- Las metaheurísticas-P se diferencian en la forma en que se implementan los procedimientos de generación y selección, y en el tipo de memoria que se usa durante la búsqueda.

Metaheurística-P

Memoria de Búsqueda

Algoritmos evolutivos

Colonia de hormigas

Optimización por enjambre de partículas

Colonia de abejas

Sistemas inmunes artificiales

Población de individuos

Matriz de feromonas

Población de partículas

Población de abejas

Población de anticuerpos

Más conceptos

- Memoria de búsqueda. Representa el conjunto de información extraído y memorizado durante la búsqueda.
- Generación. Se genera una nueva población de soluciones
- Selección. El último paso en una metaheurística-P consiste en elegir una nueva solución a partir de la unión de la población actual con la población generada

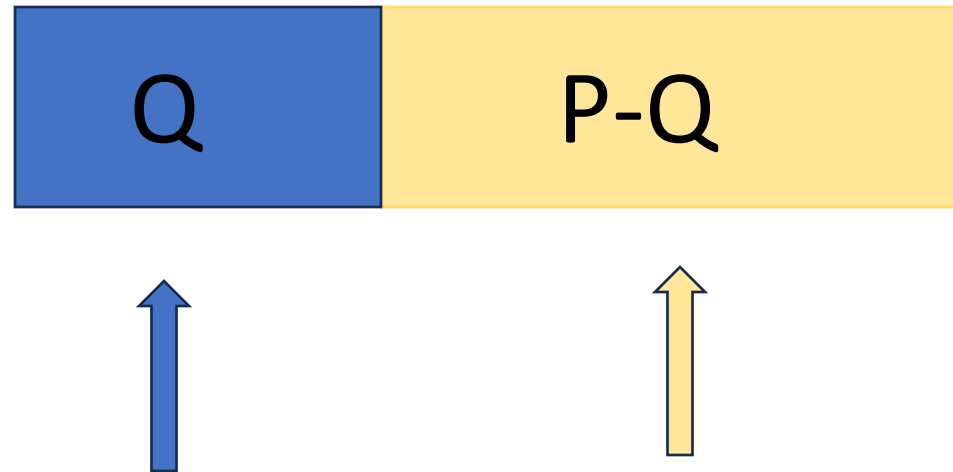
Población inicial

- La determinación de la población inicial juega un rol clave en la efectividad del algoritmo y en su eficiencia.
- El principal criterio a usar tiene que ver con la diversificación.
 - Si la población inicial no está diversificada, puede ocurrir una convergencia prematura.
- En algunas metaheurísticas, el criterio de diversificación puede tomarse en cuenta de manera explícita en la generación de la población inicial.
 - Por ejemplo, maximizando la distancia mínima entre dos soluciones cualesquiera.

Población inicial. Inicialización

- Generación random.
 - Se inicializa la población en forma aleatoria.
- Diversificación secuencial.
 - Se toman muestras elegidas de manera uniforme en el espacio de soluciones.
- Diversificación paralela.
 - La población inicial es generada en paralelo de formas independientes.
- Inicialización heurística.
 - Uso de un heurística (ej.: búsqueda local), para inicializar la población

Inicialización híbrida de una población P



Q: soluciones generadas de forma aleatoria

P-Q: soluciones generadas por diversificación secuencial

Criterio de detención

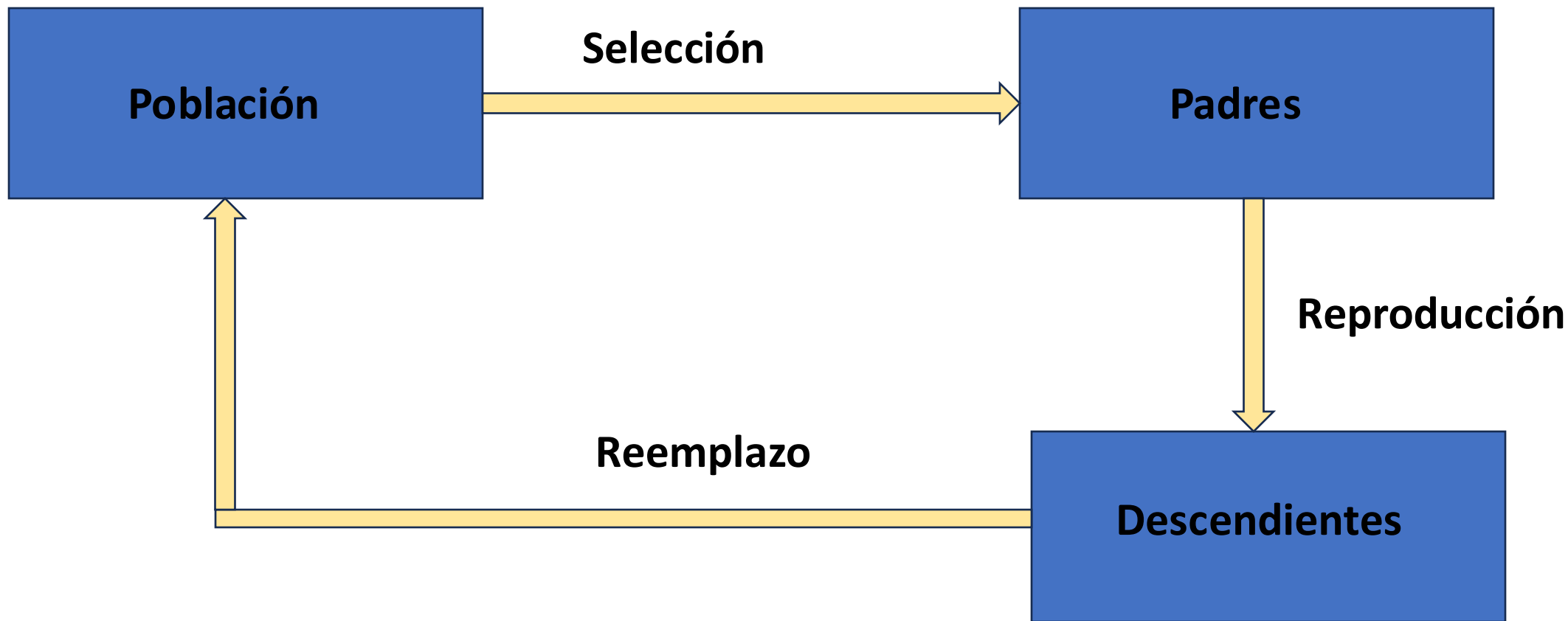
- Procedimiento estático.
 - El fin de la búsqueda puede ser conocido *a priori*. Por ejemplo, número fijo de iteraciones, límite de recursos de UCP, número máximo de evaluaciones de la función objetivo.
- Procedimiento adaptativo.
 - El fin de la búsqueda no se conoce *a priori*. Por ejemplo, se puede usar el número de iteraciones sin que haya mejoras...

Algoritmos evolutivos

- Johann Mendel (alrededor de 1850). Leyes de la herencia.
- Charles Darwin (1859). Evolución de las especies.
- Lawrence Fogel (1960). Programación evolutiva.
- John Holland (1975). Algoritmos genéticos.
- Marco Dorigo (1991). Colonias de hormigas.
- John Koza (1992). Programación genética.
- Ingo Rechenberg – H.P. Schwefel (1994). Estrategias evolutivas.
- Dervis Karaboga (2005). Colonia de abejas.
- Passino (2010). Chemotaxis (bacterias).
- Pinninghoff & Contreras (2012). Algoritmos basados en resistencia bacteriana.

Algoritmos evolutivos

- Se basan en la noción de competencia.
- Representan una clase de algoritmos para la solución iterativa de problemas de optimización, que simulan la evolución de las especies.
- En otras palabras, se basan en la evolución de una población de individuos.



Metáforas de optimización

Metáfora	Optimización
Evolución	Solución de Problemas
Individuo	Solución
Fitness	Función objetivo
Ambiente	Problema de optimización
Locus	Elementos de la solución
Alelo	Valor del elemento (locus)

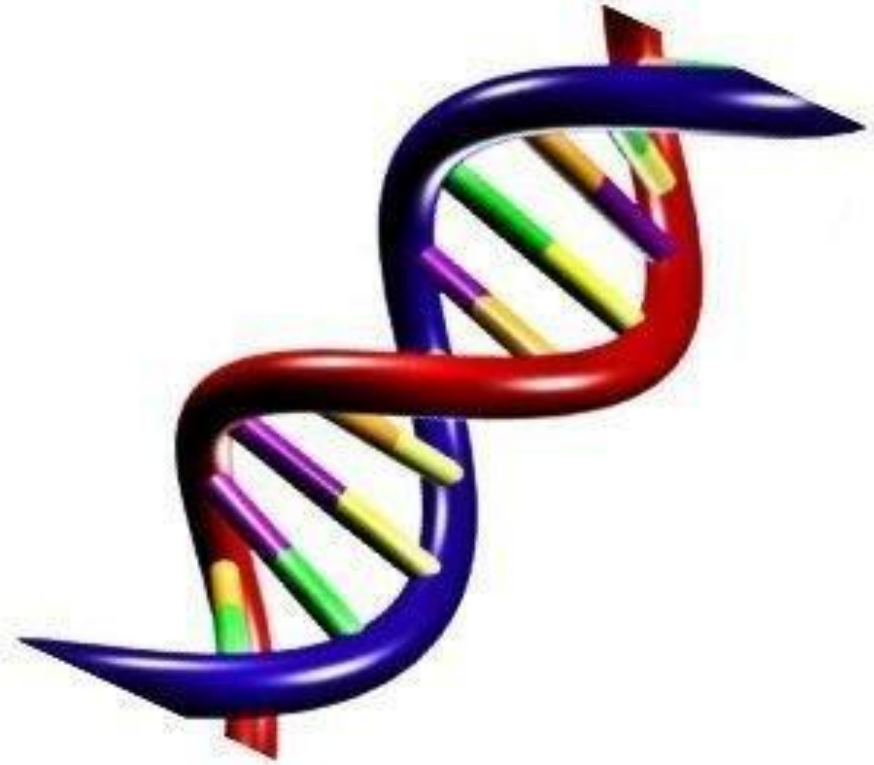
En resumen

- Soluciones computacionales optimales son intratables para muchos problemas de importancia industrial y científica.
- Las metaheurísticas representan una familia de técnicas de optimización aproximada.
- A diferencia de los algoritmos exactos, las metaheurísticas no garantizan soluciones óptimas.
- Las metaheurísticas no indican cuán cerca del óptimo está la solución obtenida.

Ahora...

- Vamos a ver metaheurísticas más concretas y más entretenidas

Algoritmos genéticos



Introducción

- Algoritmos que operan en analogía con procesos naturales
- La existencia de computadores masivamente paralelos hace que estas técnicas tengan un interés práctico
- Clave: Principio de evolución (supervivencia de los más adecuados)

En los algoritmos genéticos...

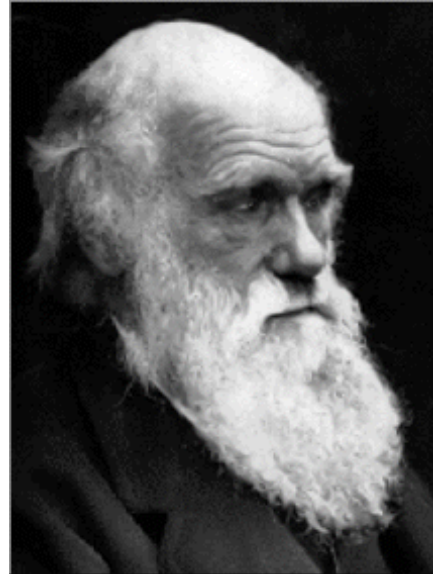
- Una población de individuos (soluciones potenciales) sufre una secuencia de transformaciones
 - Unarias (en el caso de las mutaciones)
 - De orden mayor (caso del cruzamiento) Los individuos compiten por sobrevivir

En los algoritmos genéticos...

- Los **individuos** compiten por **sobrevivir**
- Un mecanismo de **selección**, en base a ciertas características de los individuos, selecciona los individuos y **modifica la población** para formar la generación siguiente
- Después de una cierta cantidad de generaciones, el programa **converge**, en cuyo caso es de esperar que los individuos resultantes representen una **solución cercana a la óptima**

¿Qué son los AGs?

- Son algoritmos **estocásticos** cuyos métodos de búsqueda modelan un fenómeno natural: la **herencia genética** y la **evolución darwiniana** por la supervivencia.



Algunas aplicaciones de los AGs

- Optimización
- Programación Automática
- Machine Learning
- Economía
- Sistemas inmunes
- Ecología
- Genética poblacional
- Evolución y aprendizaje
- Sistemas sociales
- ...

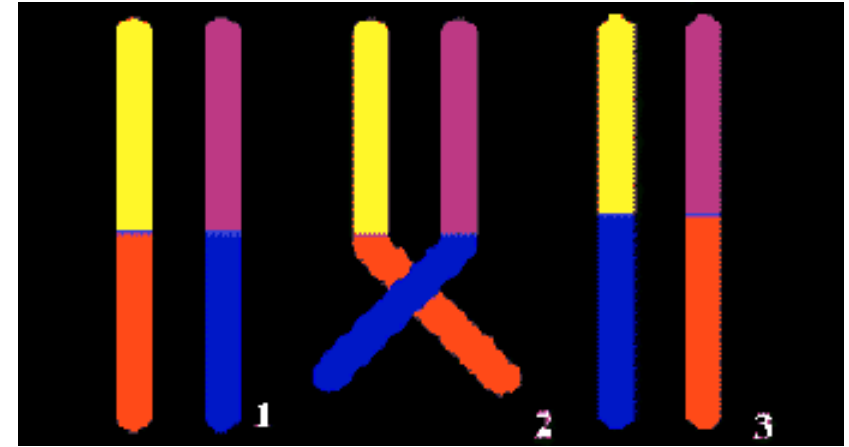
Terminología

- *Cromosoma*
 - Candidato a solución de un problema
 - A menudo se codifica como un string de bits
- *Genes*
 - Un solo bit o bloques simples de bits adyacentes que codifican un elemento particular de la solución candidata
- *Alelo*
 - 0 o 1 en strings de bits

Terminología (cont.)

- *Cruzamiento*

- Intercambio de material genético entre dos cromosomas
padres



- *Mutación*

- Cambio del valor de un gen elegido al azar

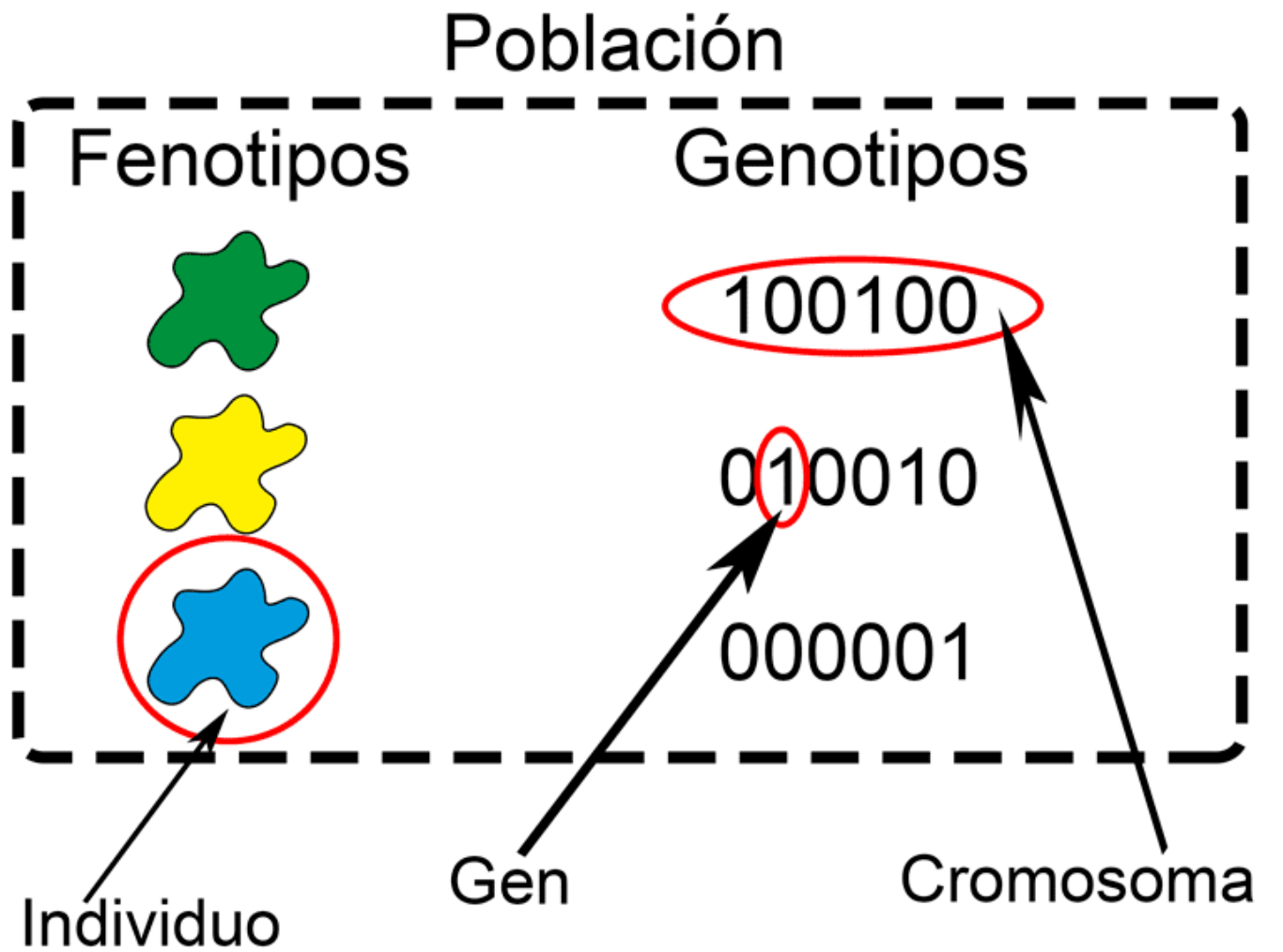
- *Genotipo*

- *Fenotipo*

- *Padres*

- *Descendientes*

Graficamente



Componentes de un AG

- Para un problema particular, un algoritmo genético debe tener los siguientes componentes

- Una **representación** genética para las potenciales soluciones al problema [**gen, cromosoma**]
- Una forma de crear una **población inicial** de soluciones potenciales [**Inicializar $P(t)$**]
- Una **función de evaluación** (como si fuera el ambiente) que valora las soluciones en función de su fitness [**Evaluación $P(t)$**]

- Operadores genéticos que alteren la composición de los hijos [Selección de padres, cruzamiento, mutación]
- Criterios de supervivencia
- Valores para los diferentes parámetros de un AG (tamaño de la población, probabilidad de aplicación de un operador genético, etc.)

Estructura de un programa evolutivo

Begin

Inicializar Población

Evaluar Población

Mientras no sea condición de término

 Seleccionar padres para reproducción

Recombinar

Cruzamiento

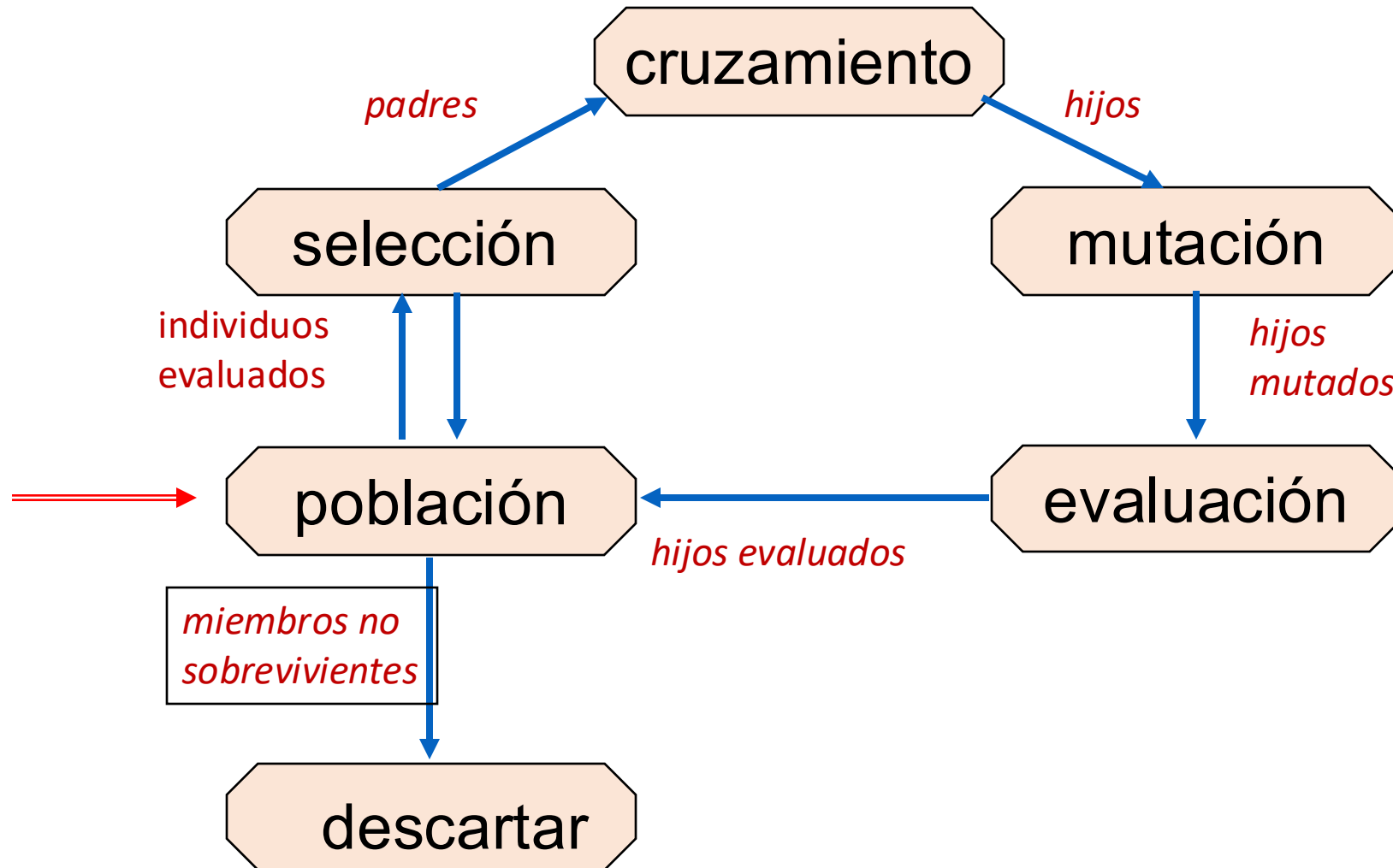
Mutación

Evaluar Población

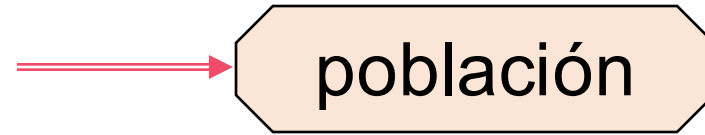
Generar nueva Población // con los que sobreviven

End

El Ciclo de Reproducción de AG



Población



Los cromosomas pueden ser:

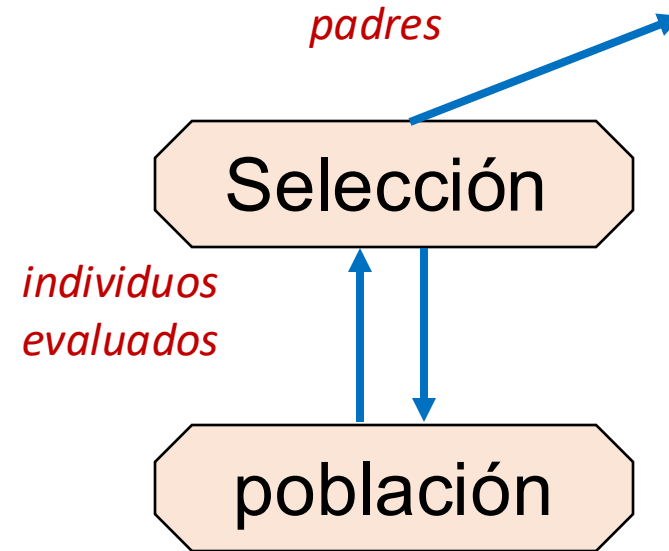
- Strings de bits (0101 ... 1100)
- Números reales (43.2 -33.1 ... 0.0 89.2)
- Permutaciones de elementos (E11 E3 E7 ... E1 E15)
- Listas de reglas (R1 R2 R3 ... R22 R23)
- Elementos de programas (genetic programming)
- ... cualquier estructura de datos...

- Un programa de evolución es un algoritmo probabilístico que mantiene una población de individuos en la iteración t

$$P(t) = \{x_i^t, \dots, x_n^t\}$$

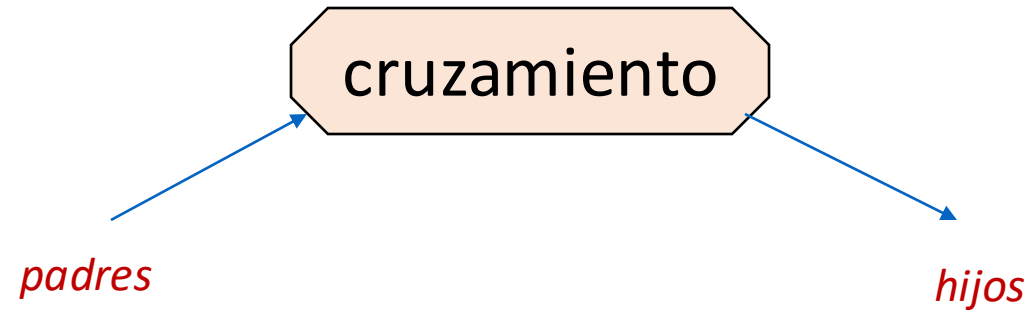
- Cada individuo representa una solución potencial al problema planteado, y es implementado como alguna estructura de datos S (posiblemente compleja)

Reproducción



Los padres son seleccionados en forma random con probabilidad de selección basada en la evaluación de los cromosomas. A mejor fitness del cromosoma, más veces puede ser seleccionado para reproducción. Se forman las parejas que se cruzarán con alguna probabilidad

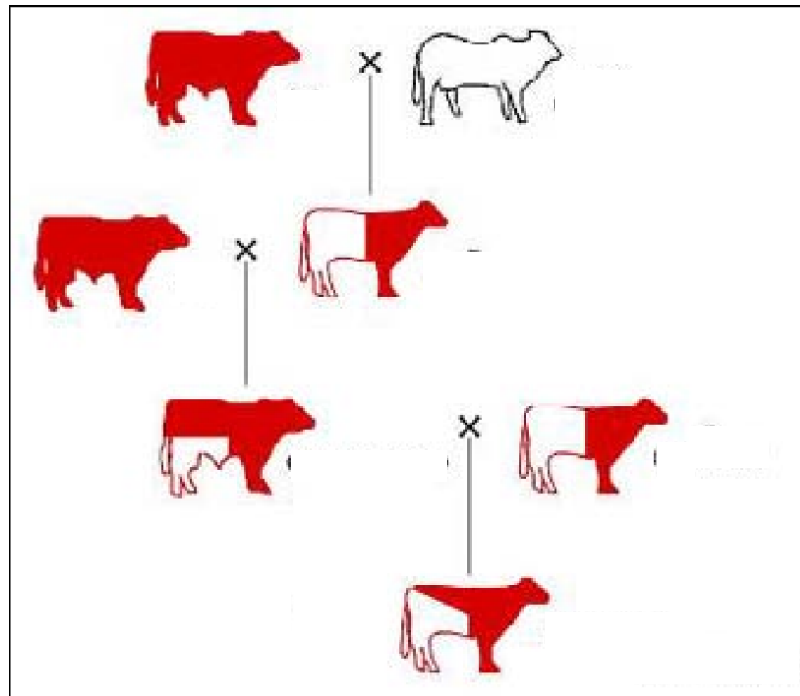
Reproducción



En el cruzamiento se elige aleatoriamente un punto del cromosoma y se intercambian los genes que están antes y después de ese punto entre dos cromosomas para generar descendientes.

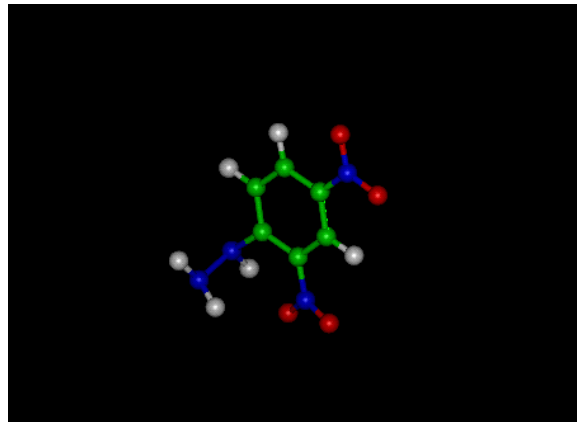
Cruzamiento: Recombinación

P1 (0 1 | 1 0 1 0 0 0) → (0 1 0 1 1 0 1 0) H1
P2 (1 1 | 0 1 1 0 1 0) → (1 1 1 0 1 0 0 0) H2

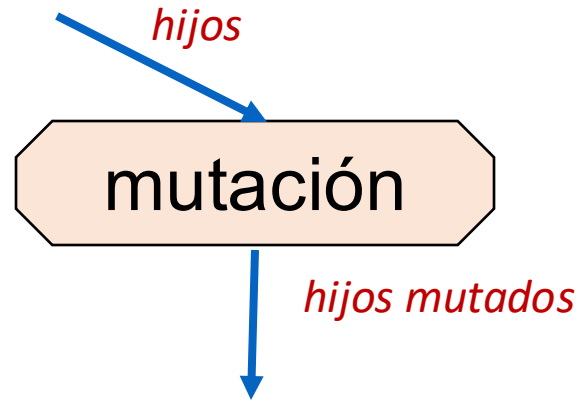


Cruzamiento

- Hay transformaciones de orden superior que recombina partes de varios (dos o más individuos) para generar uno nuevo
- $cj : s \times s \dots \times s \rightarrow s$ (crossover)



Mutación



En la mutación se produce el cambio aleatorio de un gen en un cromosoma.

Mutación: Modificación Local

Antes: (1 0 1 1 0 1 1 0)

Después: (1 0 1 0 0 1 1 0)

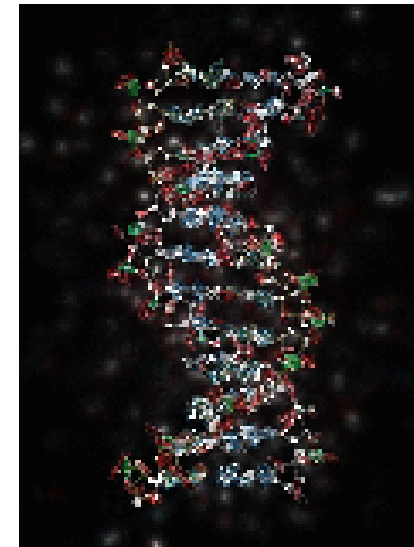
Antes: (1.38 -69.4 326.44 0.1)

Después: (1.38 -67.5 326.44 0.1)

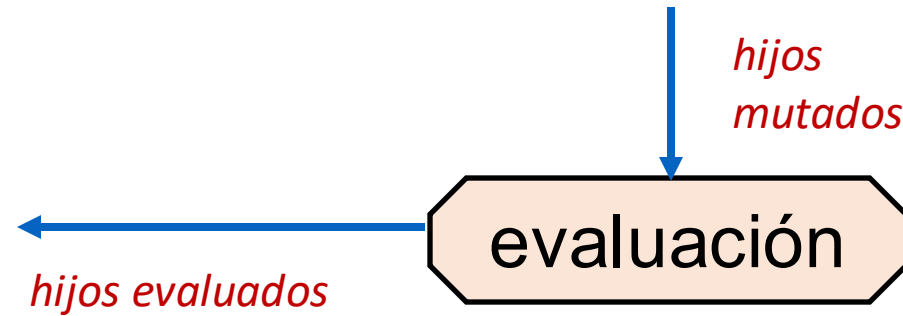
- Causa movimiento en el espacio de búsqueda (local o global)
- Puede restaurar información perdida en la población

Mutación

- Son transformaciones unarias, que crean nuevos individuos a partir de pequeños cambios en un solo individuo
- $mi : s \rightarrow s$ (mutación)



Evaluación

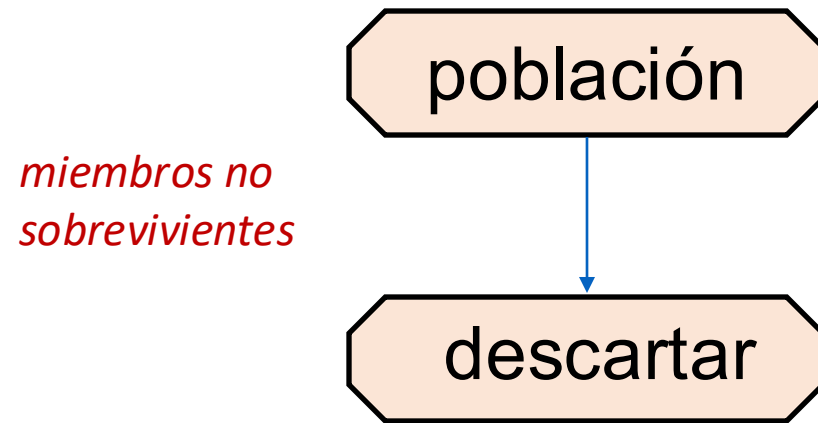


- La evaluación decodifica un cromosoma y le asigna una medida de fitness
- La evaluación es el único nexo entre un GA clásico y el problema que está resolviendo

Un vistazo al fitness

- Fitness
 - AGGMCGBL y MGGMCGBL : 1
 - AGGMCDBL y LBMPAFGA : 8

Supervivencia



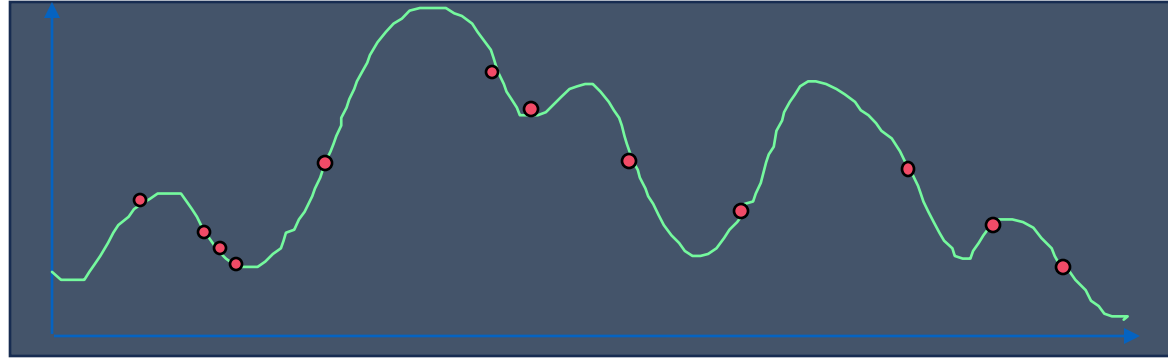
- *Reemplazo total*:
toda la población es reemplazada en cada iteración
- *Elitismo (Steady-state GA)*:
parte de la población es reemplazada en cada generación

- Se pueden formular muchos programas para un problema dado.
- Estos programas pueden ser diferentes en muchos aspectos
 - Estructuras de datos utilizadas
 - Operadores genéticos considerados
 - Métodos para el manejo de restricciones
 - Parámetros:
 - Tamaño de la población
 - Probabilidad de aplicación de operadores
 - etc.

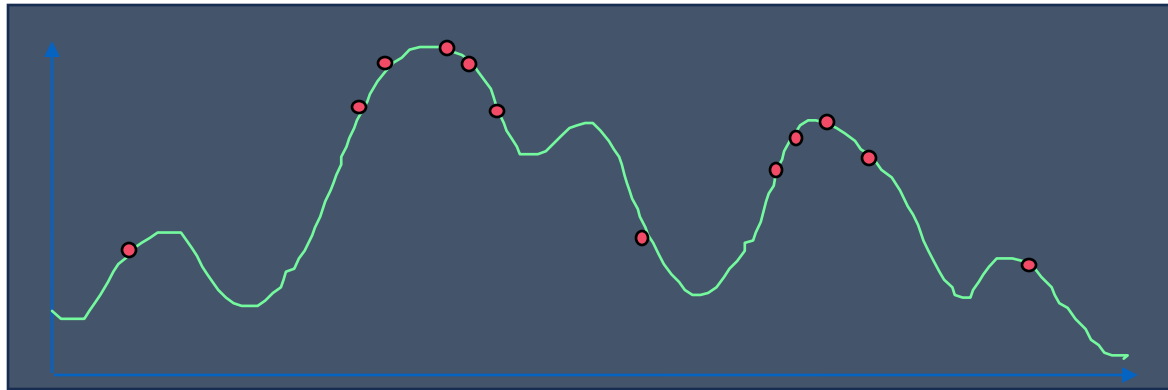
Sin embargo, la idea central
permanece

- Una población de individuos sufre transformaciones, y durante este proceso de evolución, los individuos luchan por su supervivencia.

Un Ejemplo Abstracto

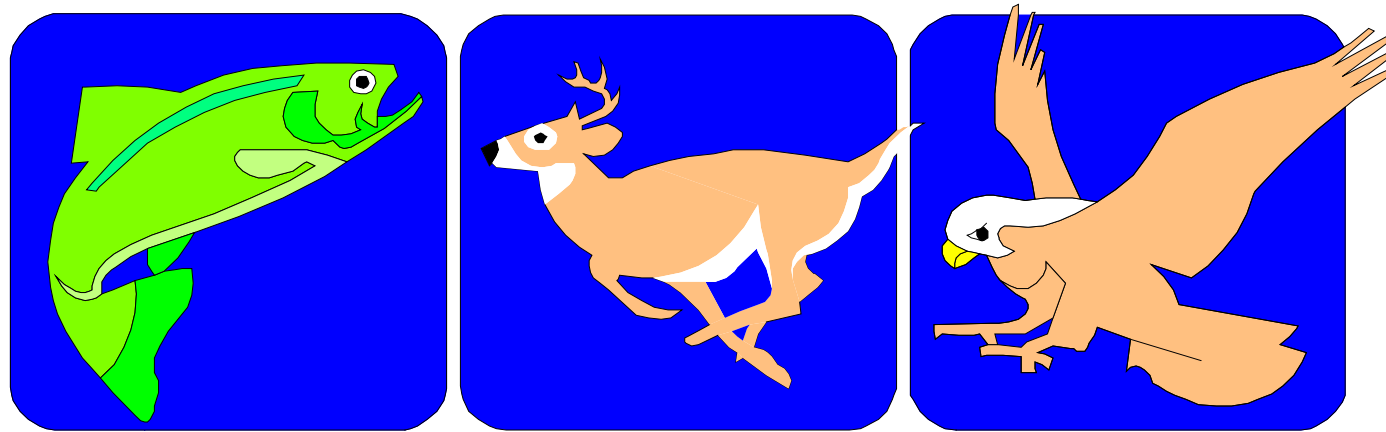


Distribución de Individuos en Generación 0



Distribución de Individuos en Generación N

Un Ejemplo Simple

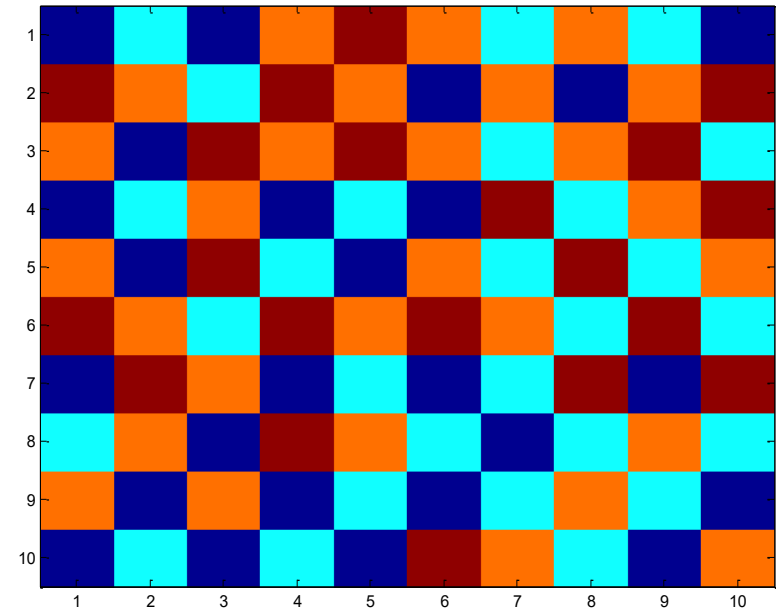
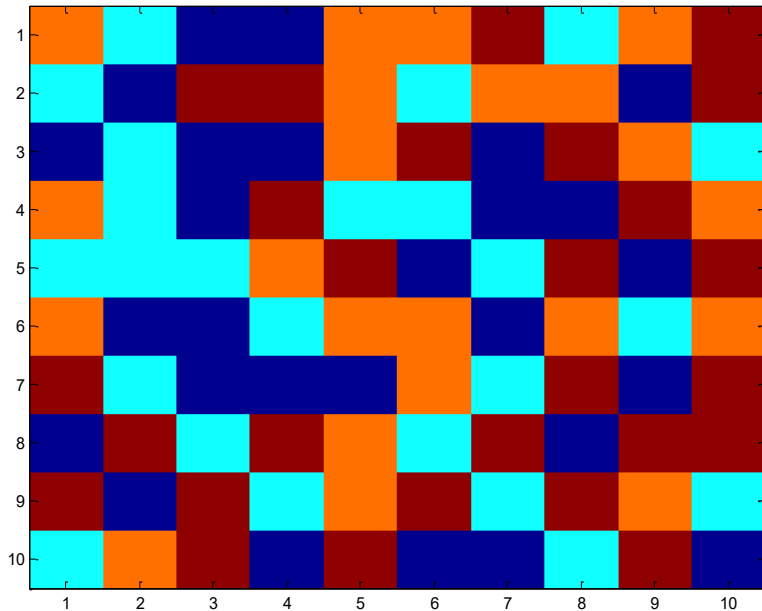


“The Gene is by far the most sophisticated program around.”

- Bill Gates, *Business Week*, June 27, 1994

Ejemplo tablero

- Tenemos un tablero de n por n espacios, en el cual cada espacio contiene un color elegido de un conjunto de cuatro colores diferentes.
- El objetivo es lograr un tablero tal que no hay vecinos con el mismo color (no diagonal)



Ejemplo tablero

- Los cromosomas representan la manera en que el tablero está coloreado.
- Los **cromosomas** no están representados por strings de bits sino por una **matriz de números**
- Los número de la matriz pueden tener uno de cuatro valores: 0, 1, 2 o 3, dependiendo del color que representan
- El **cruzamiento**, en este caso consiste en manipulación de una matriz. El cruzamiento será un intercambio de partes de la matriz, ya sea en forma horizontal, vertical, triangular o cuadrados.
- La **mutación** consiste en cambiar un número por otro.
- ¿Cuál sería el **fitness**?

Otro Ejemplo

- Supongamos que buscamos un grafo que debe satisfacer ciertos requerimientos (por ejemplo, una topología óptima de una red de comunicación o transporte).
- Cada individuo representa una solución potencial al problema, o sea cada individuo es un grafo.
- La población inicial, $P(0)$ es una población de grafos, lista para evolucionar. Normalmente, la función de evaluación es conocida y representa los requerimientos del problema.

- La función de evaluación entrega el *fitness* de cada grafo, distinguiéndose entonces los grafos peores y los mejores
- En este punto es posible considerar diversos operadores de mutación y de cruzamiento. Por ejemplo, la aparición o desaparición de una arista y la combinación de dos o más grafos.
- Si el grafo que buscamos es conexo y acíclico (o sea, un árbol), entonces las operaciones deben mantener esas características que serán consideradas en la función de evaluación.

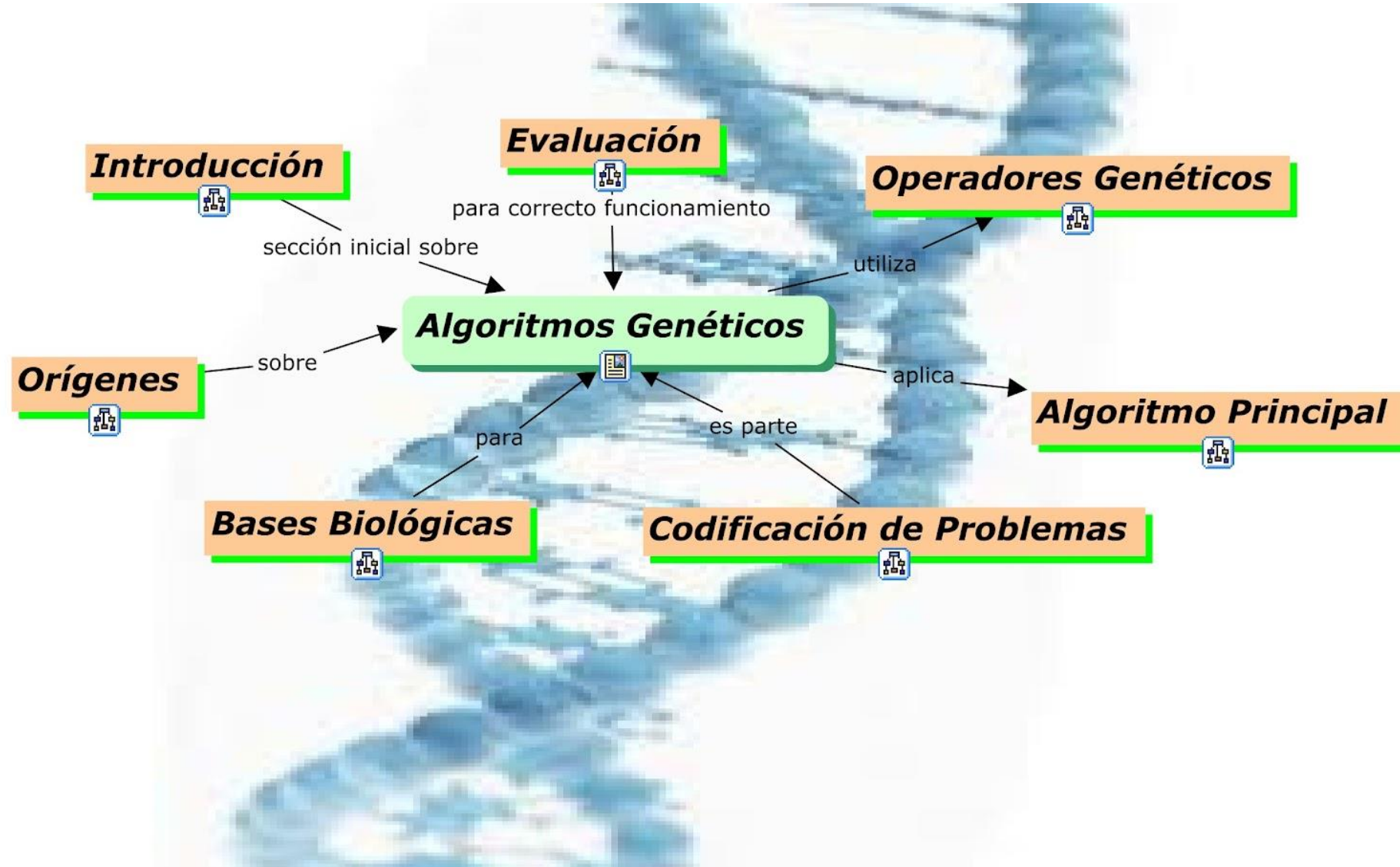
Algoritmos Genéticos

- Los AG originales de Holland son conocidos ahora como Algoritmos Genéticos Sencillos (AGS)
- Otros AGs usan diferentes:
 - Representaciones
 - Mutaciones
 - Cruzamientos
 - Mecanismos de Selección

El AG Simple

- Ha sido objeto de muchos estudios (en sus comienzos)
 - Aún es usado por personas que lo utilizan como una base para desarrollar nuevas variaciones
- Tiene algunas desventajas; por ejemplo:
 - La representación es muy restrictiva
 - Mutación y cruzamiento sólo son aplicables a strings de bits y representación de enteros
 - Mecanismos de Selección son sensibles a la convergencia de la población cuando los valores de fitness son muy cercanos
 - El modelo de población generacional puede ser mejorado con una selección de supervivencia explícita

Operadores

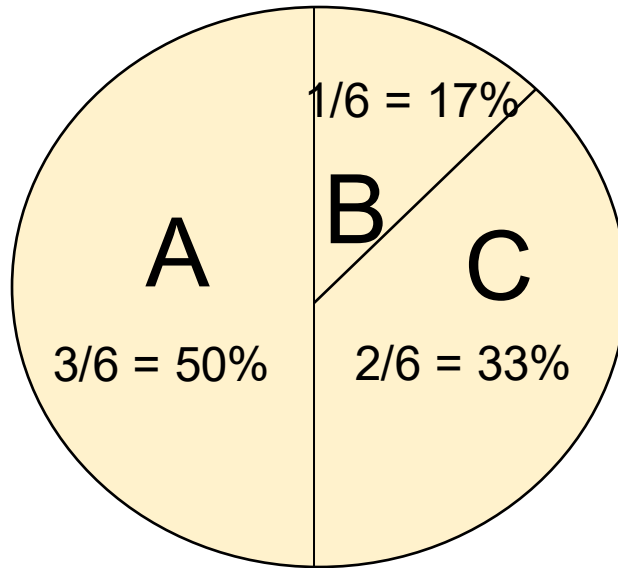


Ciclo de reproducción de AGS

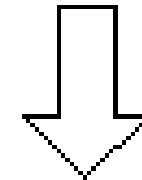
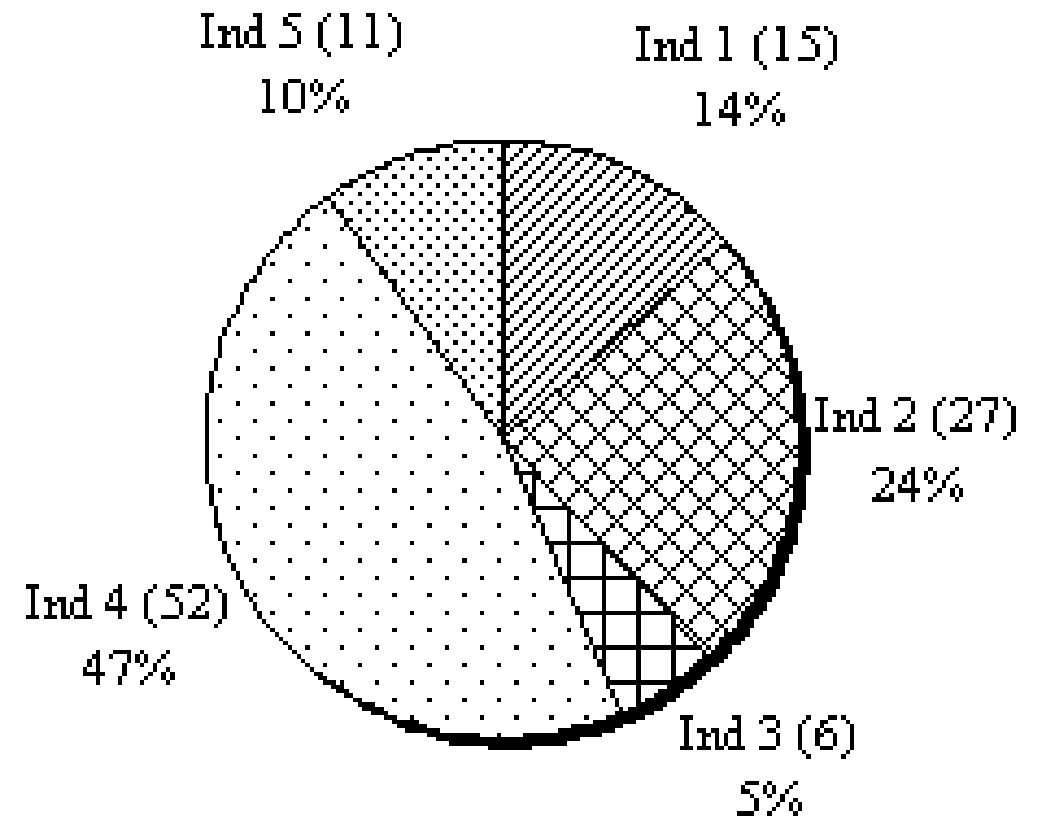
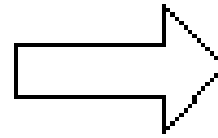
- Seleccionar los padres para el pool de apareamiento (tamaño del pool de apareamiento = tamaño de la población)
- Mezclar el pool de apareamiento
- Por cada par consecutivo aplicar cruzamiento con probabilidad p_c , de otro modo copiar los padres
- Por cada descendiente aplicar mutación a cada bit independientemente con probabilidad p_m
- Reemplace toda la población con el resultado de la descendencia

Operador de Selección

- Idea principal: individuos mejores tienen una mayor chance de ser elegidos como padres
 - Chance proporcional al fitness
 - Implementación: técnica de la rueda de ruleta
 - Asigna a cada individuo una parte de la rueda de ruleta
 - Gira la rueda n veces para seleccionar n individuos

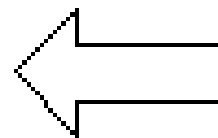


<i>Población</i>	<i>Fitness</i>
<i>Individuo 1</i>	15
<i>Individuo 2</i>	27
<i>Individuo 3</i>	6
<i>Individuo 4</i>	52
<i>Individuo 5</i>	11



Individuo 2 es seleccionado

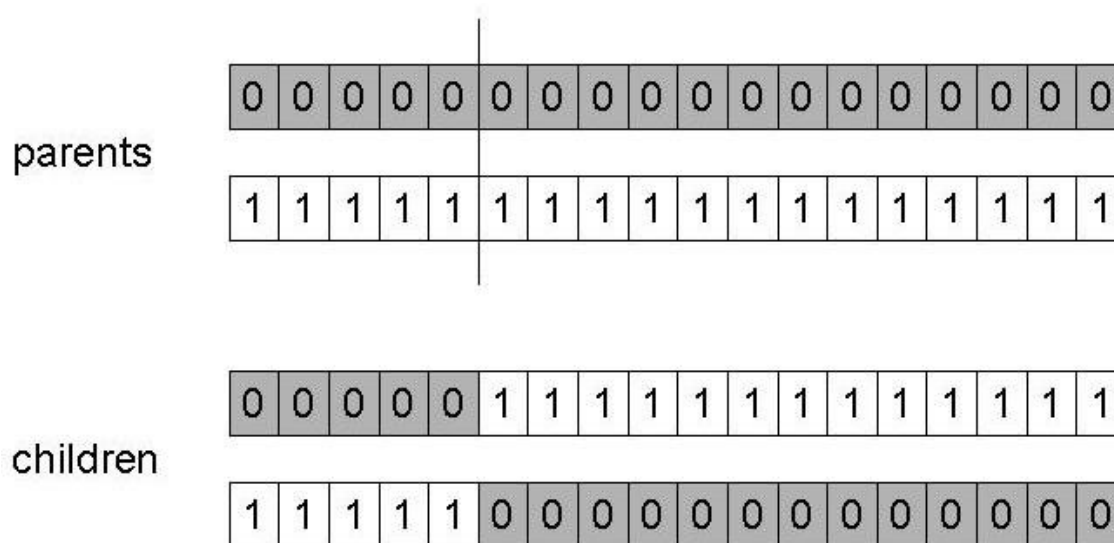
Número generado random=21



Ruleta

Operadores AGS : cruzamiento 1-punto

- Se elige un punto random (punto de cruce), igual para los dos padres
- Divide a los padres en el punto de cruzamiento
- Crea los hijos intercambiando las partes de los padres
- P_c típicamente en un rango (0.6, 0.9)

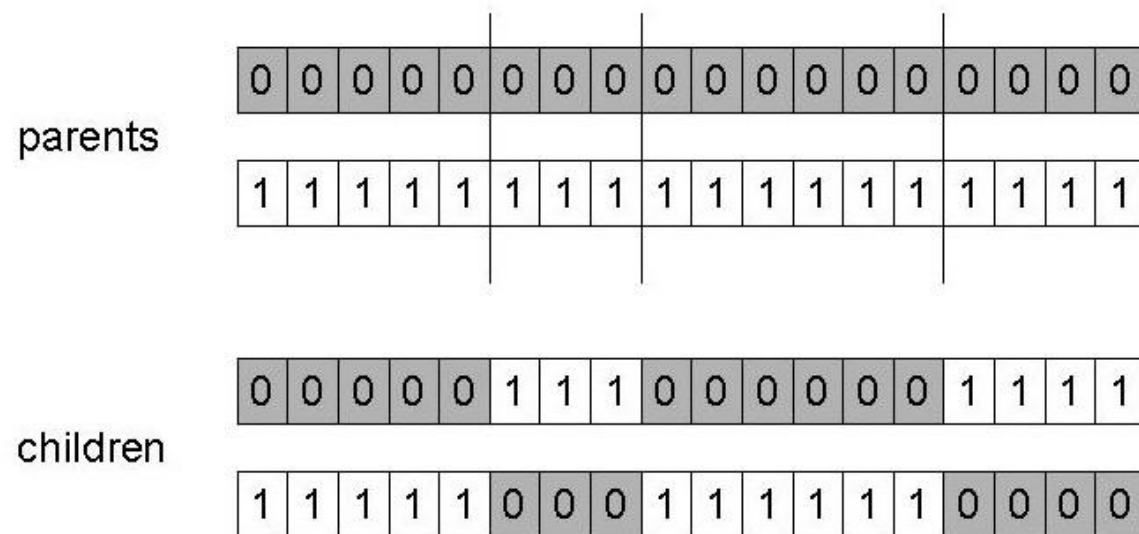


Operadores de Cruzamiento Alternativos

- El funcionamiento del cruzamiento en un punto depende del orden en que las variables aparecen en la representación
 - Mayor tendencia a mantener juntos genes que están cerca, unos de otros
 - Nunca puede mantener juntos genes de extremos opuestos
 - Esto se conoce como *Bias Posicional*
 - Puede explotarse si conocemos sobre la estructura del problema, pero usualmente ese no es el caso

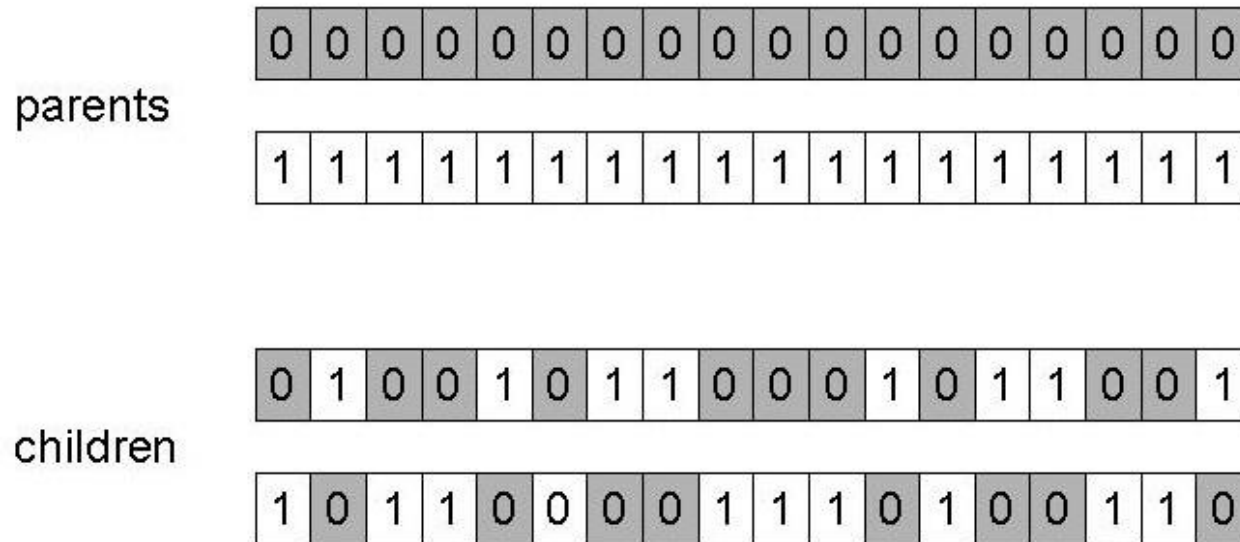
Cruzamiento en n-puntos

- Se eligen n puntos de cruzamiento en forma random
- Se hace la división en esos puntos
- Se pegan las partes resultantes en forma alternada
- Generalización de 1 punto (aún con algo de bias posicional)



Cruzamiento uniforme

- Asigna “cabezas” a un padre, “colas” al otro
- Lanza una moneda para cada gen del primer hijo
- Hace una copia inversa del gen para el segundo hijo
- La herencia es independiente de la posición



Strings Iniciales

1-Punto

11000101 01011000 01101010

00100100 10111001 01111000

2-Puntos

11000101 01011000 01101010

00100100 10111001 01111000

Uniforme

11000101 01011000 01101010

00100100 10111001 01111000

Descendientes

11000101 01011001 01111000

00100100 10111000 01101010

11000101 01111001 01101010

00100100 10011000 01111000

01000101 01111000 01111010

10100100 10011001 01101000

Operadores AGS : mutación

- Altera cada gen independientemente con una probabilidad p_m
- p_m es llamado la tasa mutación
 - Tipicamente entre $1/\text{mayor-tamaño}$ y $1/\text{largo-cromosoma}$

Padre

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Hijo

0	1	0	0	1	0	1	1	0	0	0	1	0	1	1	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Resumen AGS

Representación

Strings binarios

Recombinación

N-puntos o uniforme

Mutación

Elección de bit con
probabilidad fija
Proporcionada por fitness

Selección de padres

Hijos reemplazan padres

Supervivencia

Especialidad

Enfasis en cruzamiento

- *La metáfora subyacente en AG es la de la evolución natural. En la evolución, el problema que cada especie enfrenta es el de buscar las adaptaciones que sean beneficiosas ante un ambiente complicado y cambiante. El conocimiento que cada especie ha ganado es incorporado en la estructura cromosómica de sus miembros.*

- Después de un cierto número de generaciones, cuando no se observa un mejoramiento, los mejores cromosomas representan una solución optimal (posiblemente la global).
- A menudo el algoritmo es detenido después de un número fijo de iteraciones, dependiendo de los criterios de velocidad y recursos (computacionales).

Algunas aplicaciones

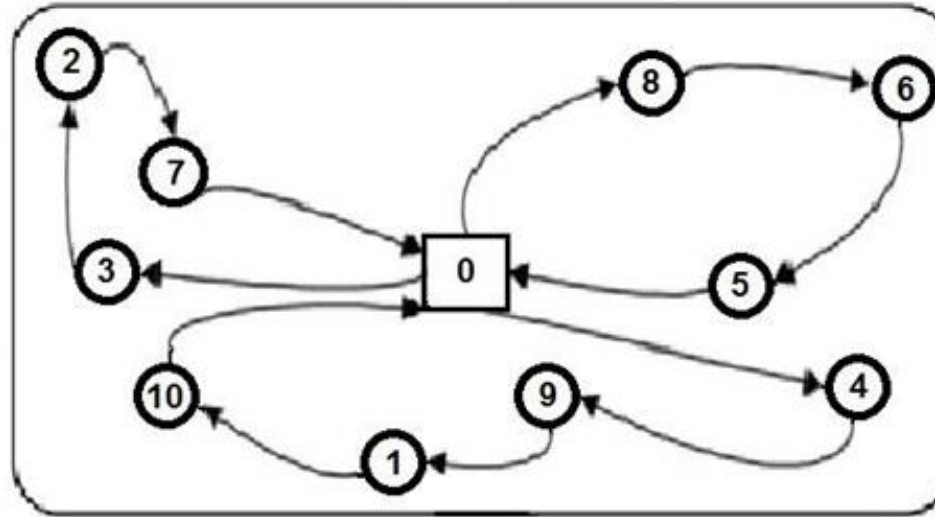
- VRP. Vehicle Routing Problem
- Bullying
- Formación de grupos
- Tratamiento de imágenes

VRP

- Es un problema de optimización combinatorial.
- Responde a la pregunta: ¿Cuál es el conjunto óptimo de rutas para una flota de vehículos que deben satisfacer los requerimientos de un conjunto de clientes?

Ejemplo

Solución



Gene



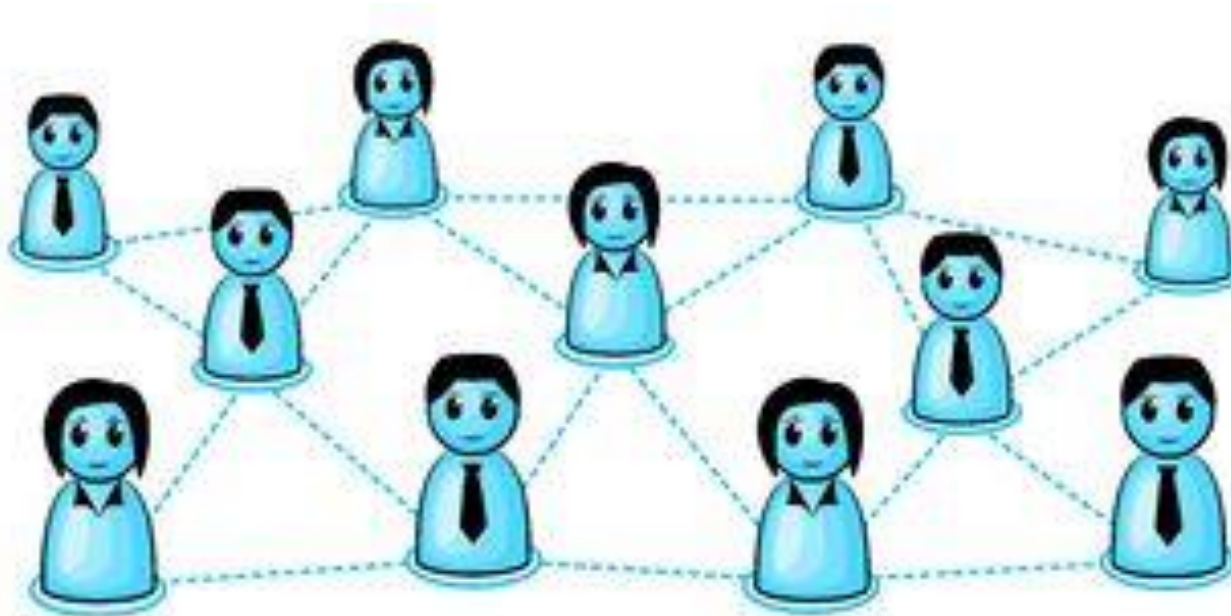
Cromosoma



Significado

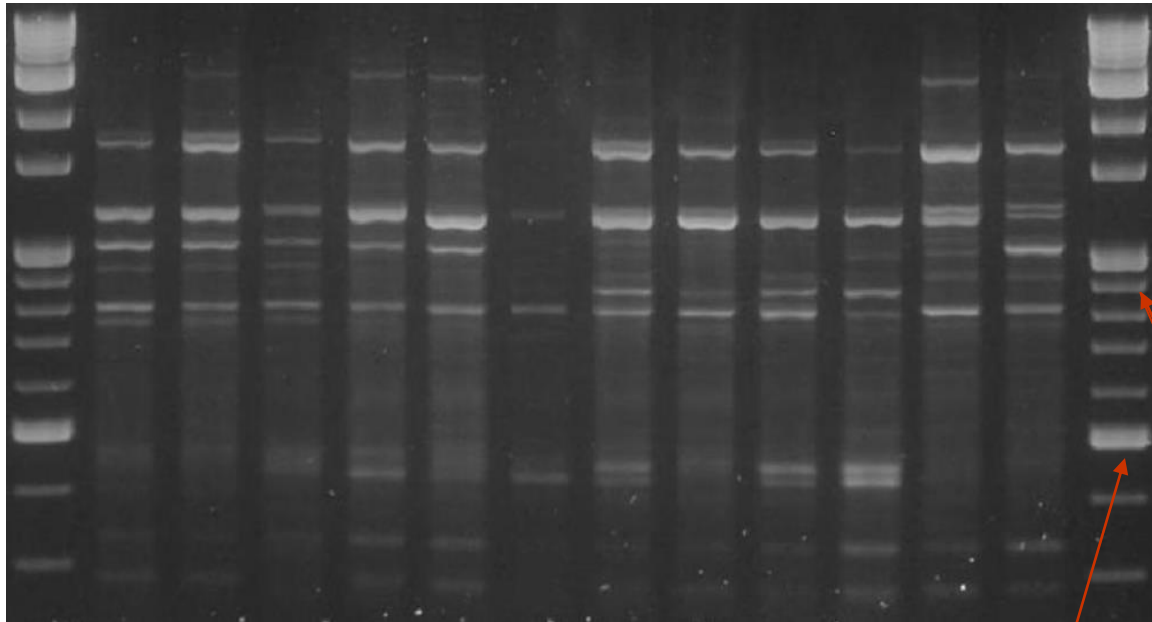
Bullying

- ¿Cómo tratamos este problema?
- Sociograma: Una representación gráfica de las relaciones sociales entre individuos.



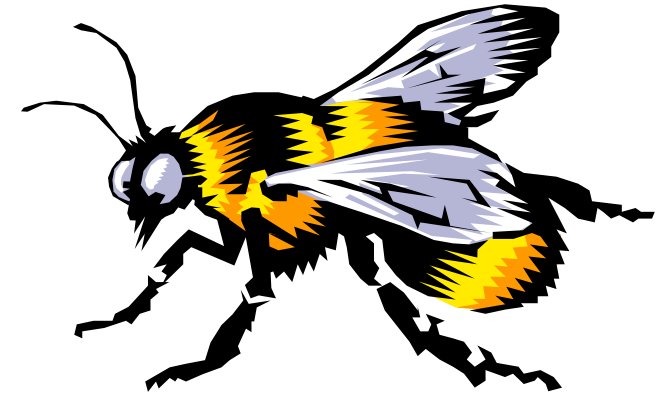
Una imagen RAPD típica

Muestras de *Eucalyptus globulus*



Carril = muestra

Banda = Peso molecular



Artificial Bee Colony (ABC) - Dervis Karaboga



Inteligencia de enjambre



- Conceptos fundamentales
 - Auto-organización
 - División del trabajo

Auto-organización



- Propiedades:
 - Retroalimentación positiva
 - Retroalimentación negativa
 - Fluctuaciones
 - Interacciones múltiples

División del trabajo



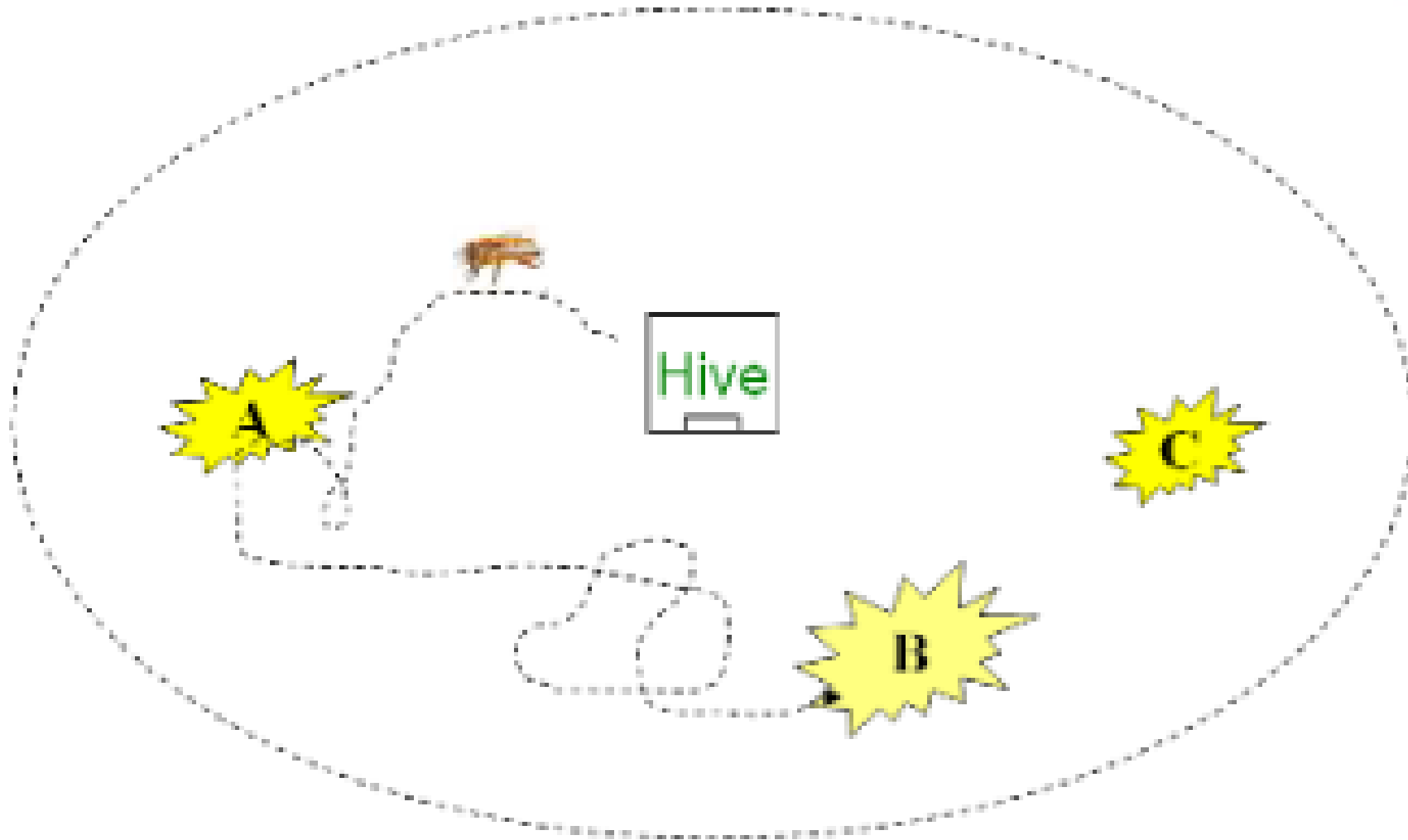
- Propiedades:
 - Tareas diferentes
 - Ejecutadas simultáneamente por individuos especializados
 - Cooperación
 - Inteligencia colectiva
 - Responde a cambios en el ambiente

Abejas naturales

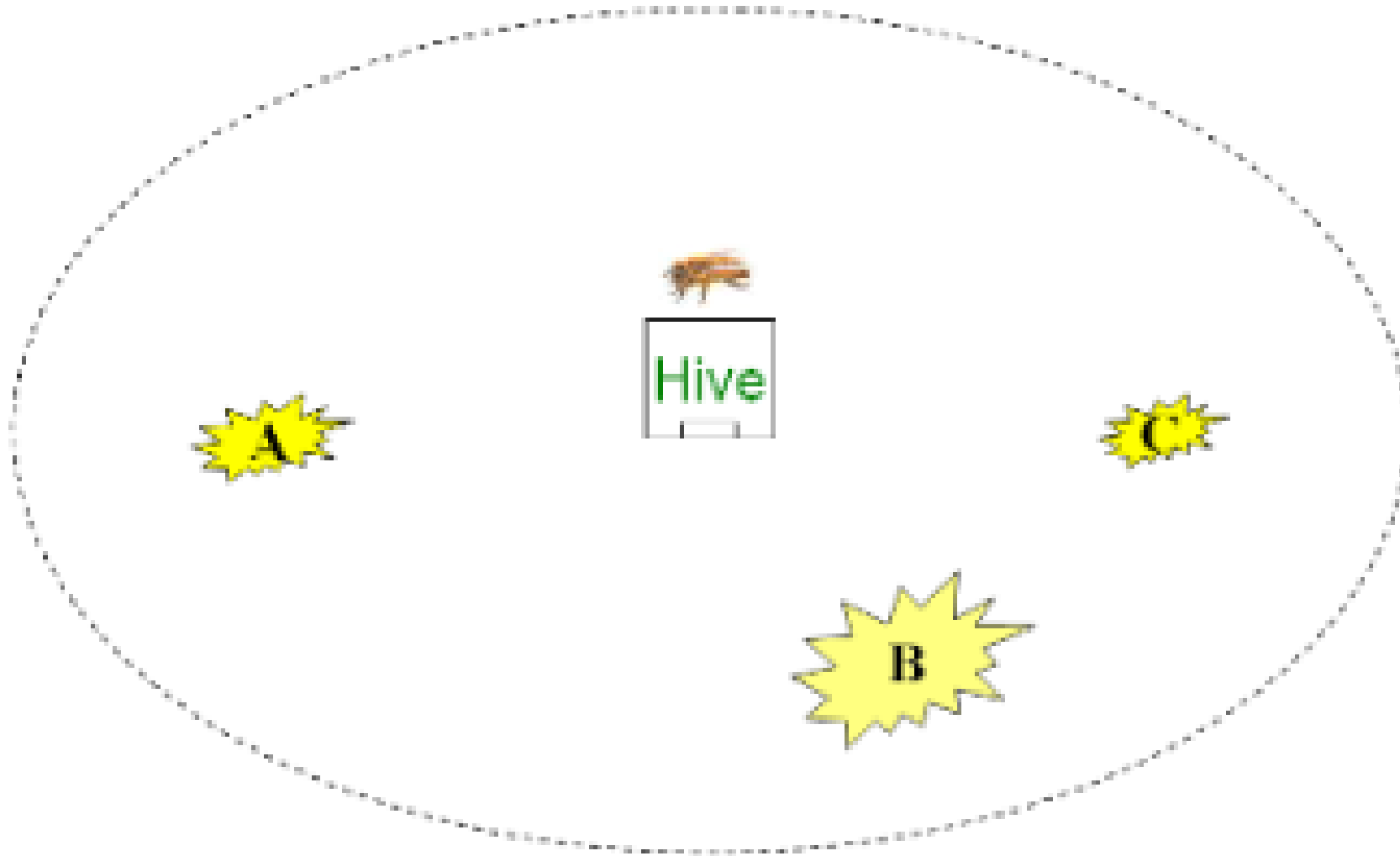


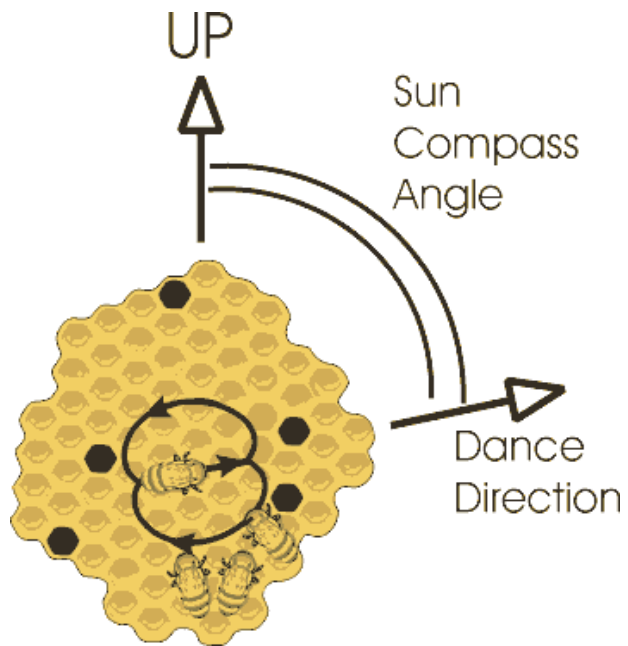
- Fuente de comida
- Abejas obreras
- Baile
- Abejas observadoras
- Abejas exploradoras

Abejas obreras exploran el vecindario

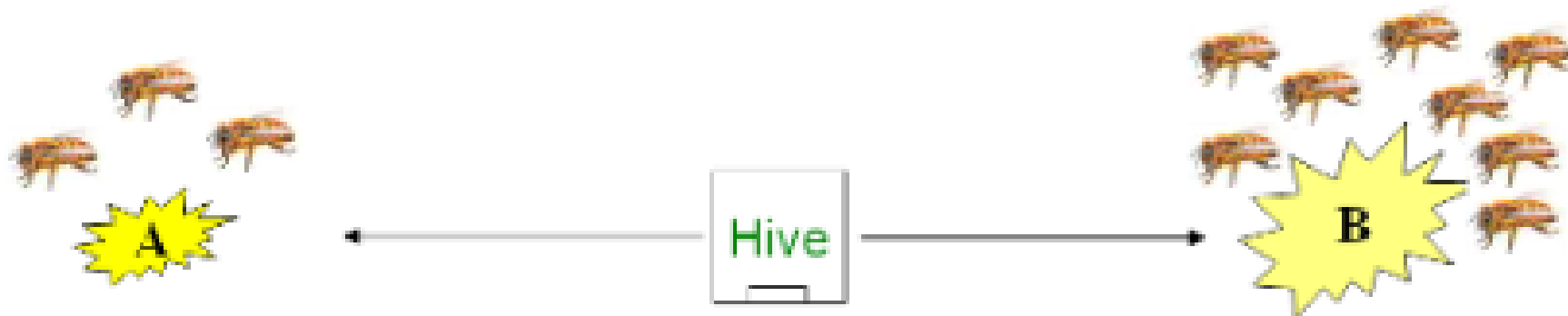


Abejas obreras retornan a la colmena
con la información encontrada





Mediante el baile informan a las observadoras la dirección y calidad de las fuentes de comida

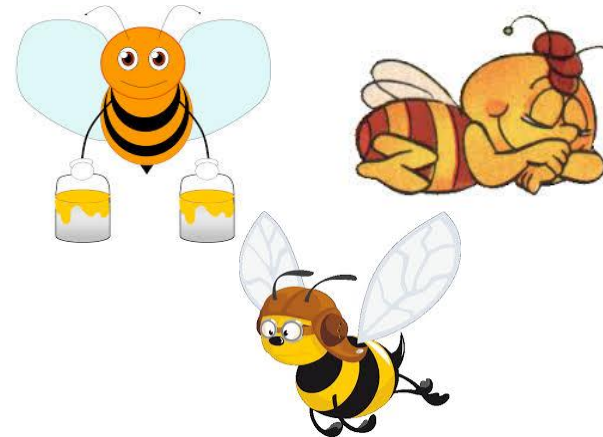


Mientras mejor es la fuente mayor es la probabilidad de que las observadoras la elijan

ABC (Karaboga 2005)



- Componentes:
 - Fuente de alimento
 - Abejas
 - Obreras
 - Espectadoras
 - Exploradoras
 - Acciones
 - Reclutamiento
 - Abandono de fuente de comida



Procedimiento



Inicializar población con soluciones random

Evaluar población

While (no se cumpla condición de término)

- ubicar obreras en fuentes de comida

- ubicar espectadoras en fuentes de comida

- enviar exploradoras a buscar nuevas fuentes de comida

- memorizar la mejor fuente de comida

end

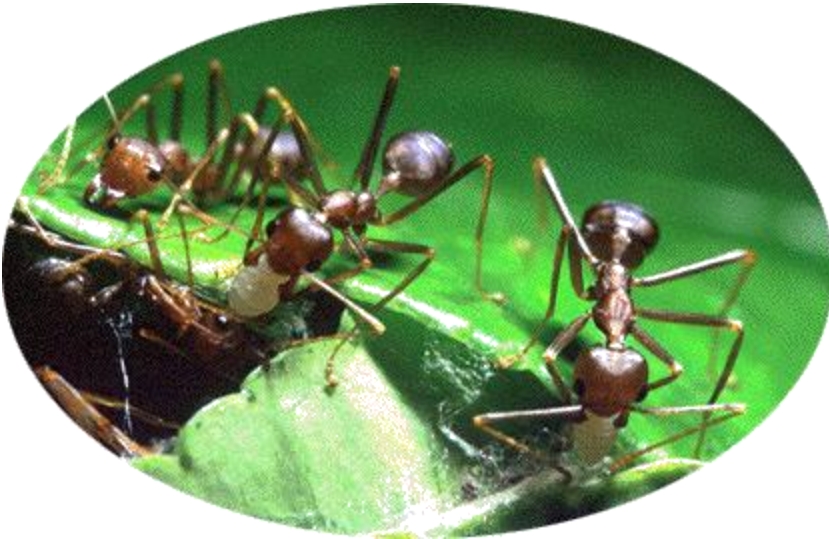
Aplicaciones



- Problemas de optimización
 - Optimización de funciones numéricas
 - Optimización combinatoria
 - Optimización con restricciones
 - Clasificación
 - Detección de bordes en imágenes
 - Etc.

Algoritmos de hormigas

Un enfoque emergente



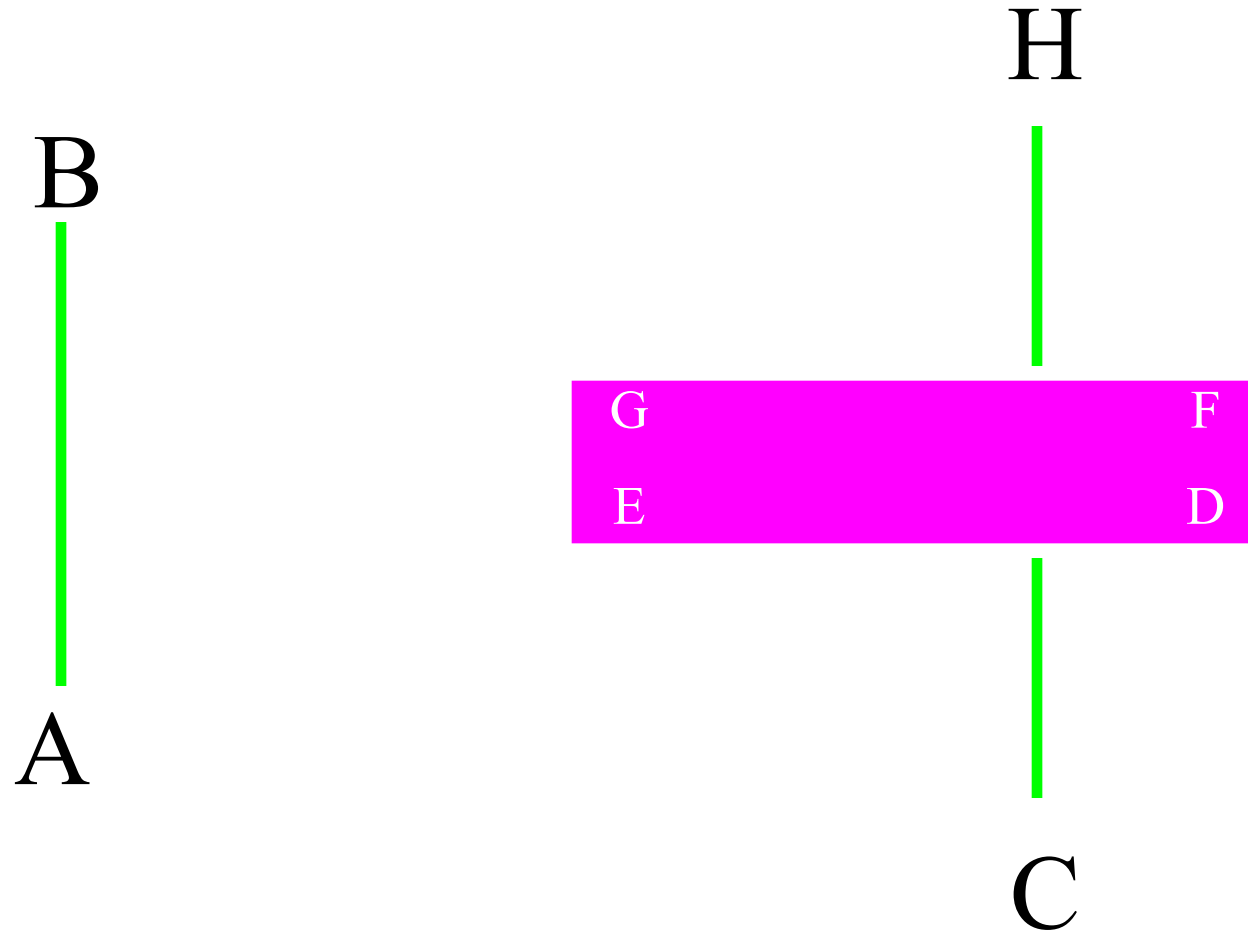
ALGORITMOS DE HORMIGAS

- Las hormigas son prácticamente ciegas, pero se las arreglan para encontrar los caminos a las fuentes de alimentación. ¿Cómo lo hacen?
- Estas observaciones inspiraron un nuevo tipo de algoritmos llamados *ant algorithms* (o ant systems).
- Dorigo, 1996

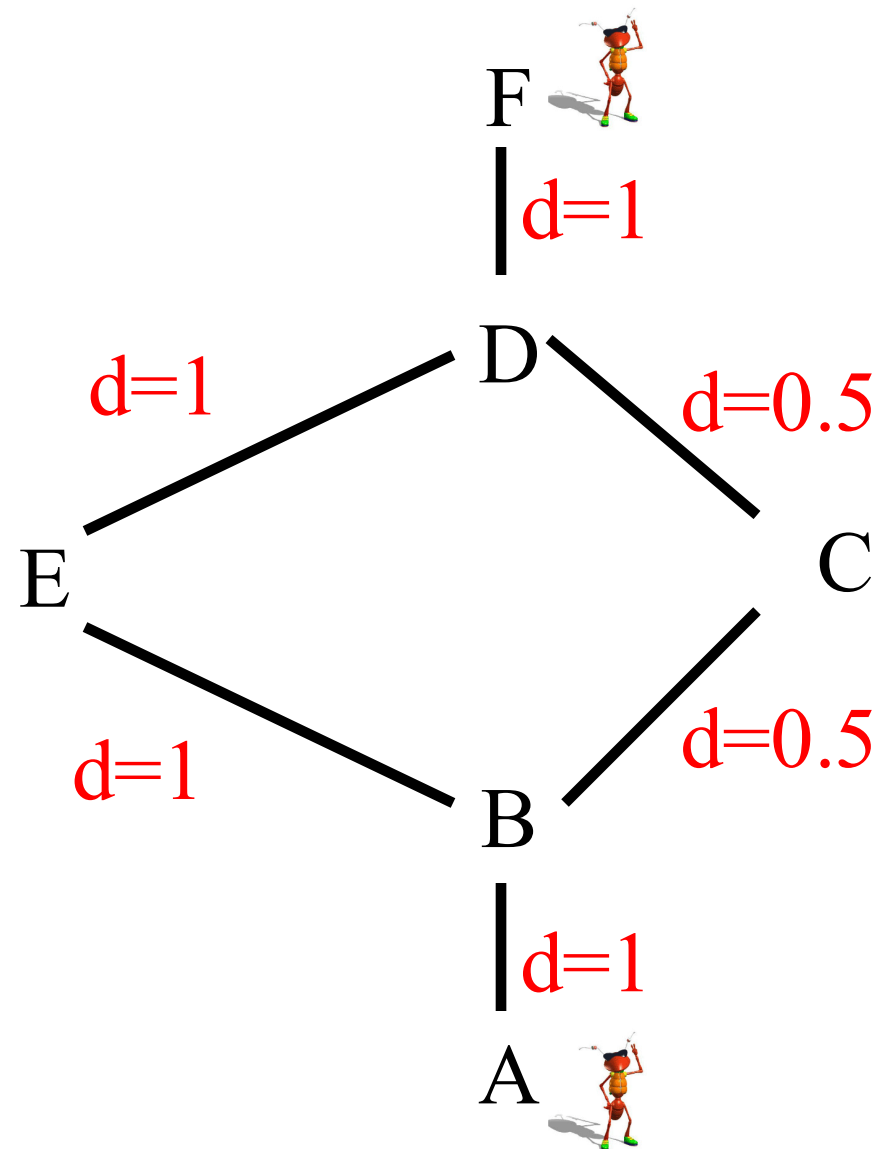
ALGORITMOS DE HORMIGAS

- Son basados en una población. En ese sentido son una aproximación o similares a los AGs.
- Existe una población de hormigas, donde cada hormiga busca una solución y luego se comunica con las otras hormigas.

ALGORITMOS DE HORMIGAS



ALGORITMOS HORMIGAS



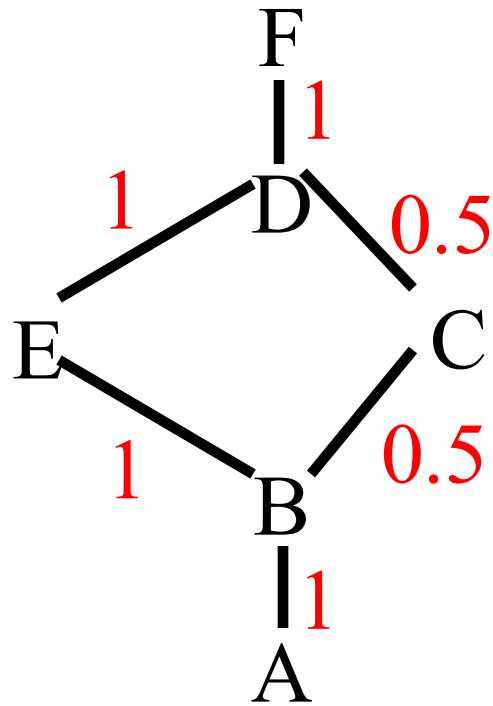
ALGORITMOS DE HORMIGAS

Tiempo, t , es discreto

En cada unidad de tiempo, una hormiga se mueve una distancia, d

Una vez que la hormiga se mueve, deposita en el camino una unidad de feromona.

En $t=0$, no hay feromona en los arcos



16 hormigas van
desde A - F y otras
16 se mueven
desde F - A

En $t=1$ habrá 16 hormigas en B
y 16 hormigas en D.

En $t=2$ habrá 8 hormigas en D
y 8 hormigas en B. Habrá 16
hormigas en E

Las intensidades en los arcos
son:

$FD = 16, AB = 16, BE = 8, ED$
 $= 8, BC = 16$ y $CD = 16$

ALGORITMOS DE HORMIGAS

- Estamos interesados en explorar el espacio de búsqueda, no simplemente encontrar una ruta.
- Debemos permitir que las hormigas exploren caminos y sigan los mejores caminos con alguna *probabilidad* proporcional a la intensidad del rastro de feromona.
- No esperamos que sigan simplemente una ruta con la mayor cantidad de feromona, ya que llegaríamos a una ruta sub-óptima

ALGORITMOS DE HORMIGAS

- La probabilidad de que una hormiga siga cierta ruta es una función, no solamente de la intensidad de feromona sino también de una función que refleja lo que la hormiga ve. (*visibilidad*)
- El rastro de feromona no es eterno..., se necesita un proceso de evaporación.

ALGORITMOS DE HORMIGAS Y EL TSP

- Al inicio del algoritmo se coloca una hormiga en cada ciudad
- Variaciones de esta estrategia fueron chequeadas por Dorigo

ALGORITMOS DE HORMIGAS Y EL TSP

- El tiempo, t , es discreto. $t(0)$ marca el inicio del algoritmo. En $t+1$ cada hormiga se habrá desplazado a una nueva ciudad
- Suponiendo que el TSP se representa por un grafo totalmente conexo, cada arco tiene una *intensidad del rastro*. Esto representa el rastro de feromona dejado por las hormigas
- Sea $T_{i,j}(t)$ la intensidad del rastro de feromona en el arco (i,j) en el tiempo t

ALGORITMOS DE HORMIGAS Y EL TSP

- Cuando una hormiga decide a qué ciudad se moverá, lo hace en base a una probabilidad basada en la distancia a esa ciudad y la intensidad de feromona en el arco que las conecta
- La probabilidad de ir a la próxima ciudad se conoce como la *visibilidad*, n_{ij} , y se define como $1/d_{ij}$, donde, d , es la distancia entre las ciudades i y j .

ALGORITMOS DE HORMIGAS Y EL TSP

- En cada unidad de tiempo se produce una *evaporación*
- La cantidad de evaporación, p , es un valor entre 0 y 1

ALGORITMOS DE HORMIGAS Y EL TSP

- Para evitar que las hormigas entren en loop, se utiliza una estructura de datos Tabu
- Esto evita que las hormigas vuelvan a pasar por ciudades que ya han visitado.
- $Tabu_k$ se define como la lista para la k -ésima hormiga y contiene todas las ciudades que ya han sido visitadas

ALGORITMOS DE HORMIGAS Y EL TSP

- Después de cada recorrido de una hormiga, se actualiza cada arco:

$$T_{ij} (t + n) = p \cdot T_{ij}(t) + \Delta T_{ij}$$

ALGORITMOS DE HORMIGAS. CASOS

- Travelling Salesman Problem (TSP)
- Ruteamiento vehicular
- Stock Cutting

ALGORITMOS DE HORMIGAS - APLICACIONES

- Marco Dorigo, mantiene una página web dedicada a este tema:

<http://iridia.ulb.ac.be/~mdorigo/ACO/ACO.html>

- Contiene información sobre algoritmos y links a los artículos relevantes publicados sobre este tema.

Próxima semana

- Metaheurísticas
 - Métodos de optimización, Simulated Annealing, Tabu Search
- Contenido opcional (algoritmos basados en bacterias)
- Control